
LogicBox

Release 300

Victor, Marine, Théodore, Timéo

Mar 27, 2026

CONTENTS:

1	LogicBox Overview	3
1.1	Vue d'ensemble	3
1.2	Architecture	3
1.3	Concepts clés	3
1.3.1	Portes	3
1.3.2	Niveaux	3
1.3.3	Systèmes UI	3
1.4	Prise en main	4
1.5	Référence API	4
2	Installation	5
2.1	Prérequis	5
2.2	Installer les dépendances	5
2.3	Lancer le projet	5
3	Utilisation du projet	7
3.1	Éditeur	7
3.1.1	Portes	7
3.1.2	Connexions	7
3.1.3	Suppression	7
3.1.4	Sauvegarde	7
3.1.5	Basculer une entrée	7
3.2	Niveau	8
3.2.1	Liste des niveaux	8
3.2.2	Joueur de niveau	8
3.3	Tutoriel	8
3.4	Options	8
4	Installation	9
4.1	Utilisation de l'IA	9
5	Rôles	11
5.1	Théodore	11
5.2	Timéo	11
5.3	Marine	11
5.4	Victor	11
6	Crédits	13
6.1	Librairies et outils	13
6.2	Textures tierces	13

7	Futur	15
7.1	Comment créer une nouvelle porte logique	15
7.2	Créer un nouveau niveau	16
8	API Reference	17
8.1	Data Module	17
8.1.1	Data Nodes	31
8.2	Engine Module	39
8.3	Logger Module	48
8.4	UI Module	49
8.4.1	UI Components	50
8.4.2	Toolbox	84
	Python Module Index	99
	Index	101

Documentation for the project.

LOGICBOX OVERVIEW

1.1 Vue d'ensemble

Ce projet fournit à l'utilisateur une simulation de portes logiques performante, Turing-complète. Le projet peut être divisé en trois parties :

- **Le Sandbox**, où les utilisateurs sont libres d'utiliser tous les outils pour laisser libre cours à leur créativité.
- **Les Niveaux**, où les utilisateurs apprennent de manière progressive et structurée les bases de la logique binaire.

L'architecture est conçue pour être extensible, permettant des portes personnalisées et plusieurs niveaux d'abstraction (1-bit, 8-bit, mixte).

1.2 Architecture

Le projet est organisé en principaux modules suivants :

- `modules.data` Modèle de domaine principal : portes, nœuds, circuits (chips) et définitions de niveaux Classes partagées et données de textures pour l'ensemble du projet
- `modules.engine` Logique de simulation et pipeline d'exécution
- `modules.ui` Toutes les interactions utilisateur, menu principal, éditeurs et niveaux
- `modules.logger` Outils de journalisation et de débogage

1.3 Concepts clés

1.3.1 Portes

Opérateurs logiques (AND, OR, NOT, etc.) implémentés sur différentes tailles de bits.

1.3.2 Niveaux

Préconçus par nous, ils représentent un objectif vers lequel l'utilisateur est guidé.

1.3.3 Systèmes UI

Outils pour éditer, déboguer et interagir avec la simulation.

1.4 Prise en main

Flux de travail typique :

1. Sélectionner un niveau
2. Trouver la bonne solution
3. Apprendre à créer vos propres projets
4. Construire vos propres circuits, sans limitations
5. Amusez-vous !

1.5 Référence API

Voir la section *API Reference* pour la documentation technique complète.

INSTALLATION

2.1 Prérequis

- Python 3.x
- Arcade

2.2 Installer les dépendances

```
pip install -r requirements.txt
```

2.3 Lancer le projet

```
python main.py
```


UTILISATION DU PROJET

3.1 Éditeur

3.1.1 Portes

Vous pouvez ajouter des portes à votre circuit en les faisant glisser depuis la barre du bas. Les portes sont réparties en 3 catégories :

- 1 bit (logique binaire)
- Portes personnalisées (basées sur d'autres circuits créés par l'utilisateur)
- 8 bits

Vous pouvez faire défiler la barre du bas pour afficher toutes les portes.

3.1.2 Connexions

Vous pouvez relier plusieurs portes en créant une connexion. Une connexion est créée en cliquant sur les entrées/sorties d'une porte (trous en forme de serrure en bas d'une porte). Une connexion ne peut être reliée qu'à des entrées/sorties de même taille en bits (1 bit avec 1 bit, 8 bits avec 8 bits). Une connexion transmet uniquement des Sorties -> Entrées. En cas de conflit, les sorties connectées en premier sont retenues comme solution finale.

3.1.3 Suppression

Vous pouvez utiliser la touche Retour arrière (Suppr) sur n'importe quelle porte ou connexion dans l'éditeur pour les supprimer. Supprimer une porte supprimera également toutes les connexions liées à cette porte. Une suppression ne peut pas être annulée, veuillez supprimer avec précaution.

3.1.4 Sauvegarde

Vous pouvez sauvegarder votre circuit actuel en utilisant la touche "S". En appuyant dessus, vous accéderez au menu de sauvegarde, où vous pourrez entrer un nom pour votre circuit (17 caractères max.). Une fois sauvegardé, le menu vous ramènera à l'éditeur. Tous les circuits sont sauvegardés automatiquement chaque minute afin d'éviter toute perte en cas de crash du jeu.

3.1.5 Basculer une entrée

Vous pouvez modifier une entrée en survolant la porte et en appuyant sur la touche "E". Si l'entrée est de 1 bit, sa valeur sera simplement inversée : Activé -> Désactivé, Désactivé -> Activé. Si l'entrée est supérieure à 1 bit, un nouvel écran apparaîtra, similaire à l'écran de sauvegarde, où il vous sera demandé d'entrer une nouvelle valeur. Pour 8 bits, la valeur est comprise entre 0 et 255. Appuyer sur "OK" vous ramènera à l'éditeur. Appuyer sur "Shift" en même temps que "OK" assignera une valeur aléatoire, dans la plage autorisée, à l'entrée.

3.2 Niveau

3.2.1 Liste des niveaux

Une liste de tous les niveaux inclus dans le jeu. Ils sont tous numérotés de 0 à 35, mais l'ordre est seulement indicatif ; vous êtes libre d'essayer n'importe quel niveau, même si vous n'avez pas terminé le précédent. Les niveaux sont également colorés selon leur difficulté : Vert -> Jaune -> Orange -> Rouge -> Violet -> Noir.

3.2.2 Joueur de niveau

Lorsque vous jouez à un niveau, trois éléments vous sont fournis :

- Une limite de temps, définie par l'auteur du niveau. Elle ne vous fera pas échouer le niveau, mais vous empêchera d'obtenir 3 étoiles.
- Les portes à utiliser, dans la barre du bas. Vous êtes limité à ces portes uniquement.
- Une table de vérité, montrant la solution attendue selon les entrées, ainsi que ce que votre circuit produira.

Pour mettre à jour la table de vérité, vous pouvez cliquer sur **“CHECK”** ; cela déclenchera également l'animation de victoire si nécessaire. Chaque niveau a une solution, et la réponse peut être affichée en appuyant sur **“ANSWER”**, ce qui validera aussi le niveau avec seulement 1 étoile. Dans un niveau, vous ne pouvez pas supprimer les entrées/sorties, seulement les déplacer.

3.3 Tutoriel

Le tutoriel vous fournit des bases sur le fonctionnement du jeu, tout comme ce document, avec un rappel du fonctionnement de chaque porte logique de base et de leur table de vérité.

3.4 Options

Les options vous permettent d'ajuster certains paramètres du jeu. Les paramètres liés à la vidéo, comme les FPS et le plein écran, nécessiteront un redémarrage complet du jeu pour être appliqués. Les paramètres sont enregistrés dans le fichier **preferences.json**, à la racine du projet.

INSTALLATION

4.1 Utilisation de l'IA

Le code présent dans ce projet a été rédigé à **99,9 %** par des élèves de NSI, en chair et en os. Cependant, l'erreur étant humaine, la solution à tous nos problèmes n'est pas toujours aussi évidente qu'on le souhaiterait.

Important

Aucun segment de code n'a été généré à 100 % par une intelligence artificielle.

Toutefois, certaines parties ont été modifiées ou reformulées à l'aide d'outils d'IA pour :

- **Résoudre des bugs** complexes ;
- **Optimiser** certaines fonctions critiques.

La principale intervention de l'IA se situe dans le **moteur logique du jeu**. Cette utilisation a permis une optimisation du code, permettant ainsi une augmentation des performances générales du jeu.

5.1 Théodore

Game designer principal du jeu. Il est responsable de la majorité des portes logiques et de leurs tables de vérité, ainsi que de l'ensemble des niveaux du jeu et de leurs solutions.

5.2 Timéo

Responsable du play-test. En tant que testeur en chef, il est capable de déceler tous les bugs et de les corriger, même dans les recoins les plus sombres du projet. Il est également responsable de plusieurs menus, notamment le système de sauvegarde des puces.

5.3 Marine

Responsable graphique. Elle s'est chargée de l'ensemble des textures du jeu (sauf mention contraire) : des boutons aux titres, en passant par les bordures. Elle a également conçu de nombreux menus, dont le tutoriel et les options.

5.4 Victor

Chef logic. Il est responsable du cœur logique du jeu et des règles de la simulation. Il a également développé les menus les plus complexes, tels que l'éditeur et l'interface de résolution des niveaux.

CRÉDITS

6.1 Librairies et outils

- **Arcade Python** : © Copyright 2025, Paul Vincent Craven.
- **Line Profiler** : Copyright (c) 2008, Enthought, Inc.
- **Black** : Copyright © 2018-Présent, Łukasz Langa et les contributeurs de Black.
- **Ruff** : Copyright (c) 2022, Charles Marsh.

6.2 Textures tierces

- **Logic Gates Textures** : © 2026 [Naked-on-the-Bus](#)

Conseils et instructions pour les futurs développeurs qui reprendront ce projet.

7.1 Comment créer une nouvelle porte logique

Créer une porte logique “native” permet d’optimiser les performances par rapport aux portes personnalisées (*custom gates*). Voici les étapes pour intégrer une nouvelle porte :

1. **Créer un nouveau fichier** dans `data/nodes/`. Il n’est pas nécessaire de le placer dans l’un des sous-dossiers existants ; ceux-ci servent uniquement à l’organisation.
2. **Éditer le fichier** pour définir les propriétés de la porte :

```
self.name: str = "AND"           # Nom affiché visuellement
self.type: str = "Gate"         # Type d'entité (ne pas modifier)
self.gate_type: str = "AND"     # Identifiant unique de la porte

self.inputs: list[int] = [0, 0] # Nombre et valeurs par défaut des entrées
self.outputs: list[int] = [0]   # Nombre et valeurs par défaut des sorties
self.inputs_sizes: list[int] = [1, 1] # Taille des entrées (1bit, 4bit, 8bit, etc.)
self.outputs_sizes: list[int] = [1] # Taille des sorties (1bit, 4bit, 8bit, etc.)
```

3. **Inscrire la porte dans l’index** : Modifiez le fichier `data/gate_index.py`. Ajoutez votre porte à la liste générale (`gate_types`) et à la liste correspondante de l’éditeur (`gate_types_1` ou `gate_types_8`).

Note

À ce stade, la porte est visible dans l’éditeur, mais elle n’a pas encore de comportement logique.

4. **Définir la logique** dans le moteur (`engine/logic.py`) :

```
def gate_and(inputs: list[int]) -> list[int]:
    """
    Calcule l'opération ET (AND).

    Args:
        inputs: Liste des valeurs d'entrée.

    Returns:
        list[int]: Liste des valeurs de sortie calculées.
    """
```

(continues on next page)

(continued from previous page)

```
# On multiplie par 1 pour convertir le booléen en entier (0 ou 1)
return [(inputs[0] and inputs[1]) * 1]
```

5. **Lier la fonction à l'index logique** (engine/logic.py) : Ajoutez votre fonction au dictionnaire LOGIC_MAP pour que le moteur puisse l'appeler :

```
LOGIC_MAP: dict[str, callable] = {
    "AND": gate_and,
}
```

6. **Profitez !** Bravo, vous venez d'ajouter votre première porte logique native !

7.2 Créer un nouveau niveau

Il est possible d'accéder au menu d'édition des niveaux en maintenant la touche **Shift** (ou **Maj**) tout en cliquant sur le bouton **"Level"**.

Voici la marche à suivre pour intégrer un nouveau défi :

1. **Conception** : Créez votre propre niveau dans l'éditeur et n'oubliez pas d'y ajouter une solution valide.
2. **Sauvegarde** : Appuyez sur la touche **"S"** pour sauvegarder. Le système vous demandera alors : * Un numéro de niveau ; * Une difficulté ; * Un temps limite.
3. **Personnalisation** : Pour modifier le nom et la description du niveau, vous devez ouvrir le fichier JSON généré dans un éditeur de texte (ex: VS Code, Notepad++) et modifier les champs suivants situés à la fin du fichier :

```
{
  "name": "8 Bits Adder",
  "description": "Add two 8-bit inputs using breaker, maker, and other gates. Output is_
↪SUM."
}
```

Tip

Assurez-vous que le format JSON reste valide après vos modifications (attention aux virgules et aux guillemets).

API REFERENCE

Table of Contents

- *Data Module*
 - *Data Nodes*
- *Engine Module*
- *Logger Module*
- *UI Module*
 - *UI Components*
 - *Toolbox*

8.1 Data Module

`class modules.data.Audio`

`class modules.data.COLORS`

Définit les constantes de la palette de couleurs au format hexadécimal.

`VALUE_OFF: str = 'D9D9D9'`

`VALUE_ON: str = 'DC2626'`

`class modules.data.Data`

Répertoire principal pour l'état global de l'application, les paramètres et les registres d'objets.

`IMAGE: ImageBuffer`

`LEVEL_BUTTONS: LevelButtonsBuffer`

`UI_EDITOR_GRID_SIZE: int`

`VERSION: int`

`WINDOW_FRAMERATE: int`

WINDOW_FULLSCREEN: bool

WINDOW_HEIGHT: int

WINDOW_WIDTH: int

categories: List[str]

current_lang: str

level_colors: List[str]

load()

Return type

None

loaded_chips: Dict[str, Any]

loaded_levels: Dict[str, Any]

save()

Return type

None

window: Any | None

class modules.data.**ImageBuffer**

Gère le stockage et le suivi de l'état d'achèvement des textures liées aux portes logiques.

add_gate_type(*id*)

Enregistre une nouvelle catégorie de porte dans le tampon.

Args:

id: Identifiant du type de porte.

Parameters

id (*str*)

Return type

None

add_texture(*id*, *texture_id*, *texture*)

Stocke un objet de texture pour une porte spécifique.

Args:

id: Identifiant de la porte. *texture_id*: Identifiant de la texture. *texture*: La ressource à stocker.

Parameters

- **id** (*str*)
- **texture_id** (*str*)
- **texture** (*Any*)

Return type

None

buffer: Dict[str, Dict[str, Any]]

complete_gate(*id*)

Marque une porte spécifique comme terminée.

Args:

id: Identifiant de la porte.

Parameters

id (*str*)

Return type

None

get_texture(*id*, *texture_id*)

Récupère une texture spécifique depuis le tampon.

Args:

id: Identifiant de la porte. *texture_id*: Identifiant de la texture.

Returns:

L'objet texture si trouvé, sinon False.

Parameters

• *id* (*str*)

• *texture_id* (*str*)

Return type

Any | bool

is_complete_gate(*id*)

Vérifie si une porte a été marquée comme terminée.

Args:

id: Identifiant de la porte.

Returns:

True si la porte est marquée comme terminée, sinon False.

Parameters

id (*str*)

Return type

bool

class modules.data.KeyBinds

class modules.data.LevelButtonsBuffer

Gère le stockage et la récupération des éléments d'interface de sélection de niveau.

buffer: Dict[str, Any]

get(id)

Récupère les données d'un bouton.

Args:

id: Identifiant du bouton de niveau.

Returns:

Les données d'image du bouton associé.

Parameters

id (*str*)

Return type

Any

set(id, image)

Stocke ou met à jour les données d'un bouton.

Args:

id: Identifiant du bouton de niveau. image: Données d'image à stocker.

Parameters

- **id** (*str*)
- **image** (*Any*)

Return type

None

class modules.data.chip.**Chip**(id)

Gère le cycle de vie, la sérialisation et la structure d'une puce logique.

Parameters

id (*str*)

copy()

Crée une copie profonde de la puce avec un identifiant nouvellement généré.

Returns:

Une nouvelle instance de Chip reflétant l'état actuel.

Return type

[Chip](#)

get_gates()

Récupère les identifiants pour tous les composants de type porte standard.

Returns:

Liste des IDs de portes standards.

Return type

List[*str*]

get_inputs()

Récupère les identifiants pour toutes les portes de type entrée (input).

Returns:

Liste des IDs de portes d'entrée.

Return type*List[str]***get_outputs()**

Récupère les identifiants pour toutes les portes de type sortie (output).

Returns:

Liste des IDs de portes de sortie.

Return type*List[str]***load()**

Construit les portes (gates) et les chemins (paths) à partir des données en tampon.

Nécessite l'exécution préalable de `partial_load`.

Return type*None***partial_load(data)**

Charge les métadonnées de base et met en mémoire tampon les données structurées.

Args:

`data`: Dictionnaire d'état brut.

Parameters

data (*Dict[str, Any]*)

Return type*None***save(no_file=False, dojson=False)**

Séréalise l'état de la puce vers un dictionnaire, une chaîne JSON ou le disque.

Args:

`no_file`: Empêche les entrées/sorties (I/O) sur le disque si vrai. `dojson`: Retourne la sortie sérialisée sous forme de chaîne JSON si vrai.

Returns:

La représentation sérialisée ou *None* si la puce a été écrite sur le disque.

Parameters

- **no_file** (*bool*)
- **dojson** (*bool*)

Return type*Dict | str | None***class modules.data.complex.Complex(id)**

Gère une porte logique capable de traiter des entrées et sorties multi-bits, incluant le rendu des étiquettes dynamiques de l'interface utilisateur pour ces signaux.

Parameters

id (*int | str*)

draw_tiles()

Affiche la texture de la porte et les étiquettes associées en fonction de l'état logique actuel.

Return type

None

setup_texts()

Initialise les objets Text de l'interface utilisateur pour toutes les broches d'entrée ou de sortie définies avec une largeur de bit supérieure à un.

Return type

None

update_text_position()

Met à jour les coordonnées du texte de l'interface utilisateur pour suivre le mouvement des zones de collision (hitboxes) de la porte.

Return type

None

update_text_readings()

Synchronise les étiquettes de texte existantes de l'interface utilisateur avec les dernières valeurs internes des signaux.

Return type

None

Fournit la classe CustomGate pour la gestion des instances de portes logiques définies par l'utilisateur.

class modules.data.custom.CustomGate(*id*, *chip*=None)

Représente une porte logique définie par l'utilisateur enveloppant une architecture de puce interne.

Parameters

- **id** (*int*)
- **chip** (*Any* | *None*)

draw_tiles()

Affiche la porte en utilisant des textures dépendantes de l'état actuel.

Return type

None

load(*data*)

Restaure l'état de la porte à partir des données de configuration fournies.

Args:

data: Dictionnaire de configuration à charger.

Parameters

data (*Dict[str, Any]*)

Return type

None

prop_io()

Propage les valeurs actuelles des entrées externes dans la structure interne de la puce.

Return type

None

save()

Sérialise l'état de la porte pour le stockage.

Returns:

Dictionnaire contenant les données de configuration spatiale et logique.

Return type

Dict[str, Any]

update_io()

Synchronise les broches d'E/S de la porte et les métadonnées du bus avec la puce sous-jacente.

Return type

None

Module de registre pour les composants de portes logiques.

Fournit une cartographie centralisée des identifiants de chaînes de caractères vers leurs implémentations de classes de portes logiques respectives pour les systèmes 1-bit, 8-bits et mixtes.

class `modules.data.gate.Gate(id)`

Gère le cycle de vie, l'état visuel et les zones de collision (hitboxes) d'interaction d'un nœud de porte logique dans l'interface de l'éditeur.

Parameters

id (*int*)

calculate_display()

Calcule les dimensions internes et actualise les zones de collision des entrées/sorties.

Return type

None

calculate_display_lite()

Effectue une mise à jour visuelle optimisée pour les scénarios de mouvement de caméra.

Return type

None

property camera: `Tuple[float, float]`

Retourne le décalage (offset) actif de la caméra.

camera_moving(*value*)

Args:

value: Coordonnées (x, y) intermédiaires pendant la transition de la caméra.

Parameters

value (*Tuple[float, float]*)

Return type

None

draw()

Exécute la logique de rendu pour la porte et les hitboxes de débogage optionnelles.

Return type

None

draw_tiles()

Effectue le rendu de la représentation graphique (texture) de l'état de la porte.

Return type

None

gen_tile_pattern()

Génère le tableau d'indices de tuiles basé sur la grille selon la configuration du nœud.

Return type

None

load(*data*)**Args:**

data: Dictionnaire d'état contenant les propriétés de la porte à restaurer.

Parameters

data (*Dict[str, Any]*)

Return type

None

property moving: str

Retourne le nom actuel de la porte.

property name: str

Retourne le nom actuel de la porte.

save()**Returns:**

Dictionnaire représentant l'état sérialisable de la porte.

Return type

Dict[str, Any]

property touched: bool | Tuple[int, int, int]**Returns:**

Soit False, soit un tuple contenant (Type de Broche, Index, Taille en Bits) où le Type de Broche 1 représente une entrée et 2 représente une sortie.

update_text_position()

Point d'ancrage pour les mises à jour spécialisées de l'alignement des étiquettes.

Return type

None

property x: float

Retourne la coordonnée x actuelle.

property y: float

Retourne la coordonnée y actuelle.

Fournit des mécanismes pour définir et gérer les éléments d'indice de l'interface utilisateur.

class modules.data.hint.**Hint**(*id*)

Représente un indice d'interface utilisateur configuré pour des formats d'affichage spécifiques.

Attributs :

chip : La représentation de données associée pour l'affichage basé sur les jetons (chips). text : La chaîne de caractères principale affichée à l'utilisateur. type : Le mode de rendu où 0 est texte seul, 1 est jeton seul, et 2 est combiné. id : L'identifiant unique de cette instance.

Parameters

id (*int*)

Fournit des classes de localisation pour les chaînes de caractères de l'interface utilisateur de l'application.

class modules.data.language.**English**

Stocke et récupère les chaînes de localisation en anglais.

get(*menu*, *key*)

Récupère une chaîne de traduction par catégorie et identifiant.

Args:

menu: Le nom de l'attribut contenant le dictionnaire de recherche. key: L'identifiant spécifique de la chaîne souhaitée.

Returns:

La chaîne correspondante si elle est trouvée, sinon un message d'erreur.

Parameters

- **menu** (*str*)
- **key** (*str*)

Return type

str

class modules.data.language.**French**

Stocke et récupère les chaînes de localisation en français.

get(*menu*, *key*)

Récupère une chaîne de traduction par catégorie et identifiant.

Args:

menu: Le nom de l'attribut contenant le dictionnaire de recherche. key: L'identifiant spécifique de la chaîne souhaitée.

Returns:

La chaîne correspondante si elle est trouvée, sinon un message d'erreur.

Parameters

- **menu** (*str*)
- **key** (*str*)

Return type

str

class modules.data.level.Level(*id*)

Gère la logique des niveaux du jeu, incluant l'état de la puce, les objectifs et la progression.

Attributs :

chip (Chip) : La puce logique associée à ce niveau. engine (Engine) : Le moteur de simulation utilisé pour évaluer la logique du circuit.

Parameters

id (*int* / *str*)

calculate_inventory()

Calcule les exigences d'utilisation des portes et l'utilisation actuelle du joueur.

Return type

None

check_victory()

Détermine si les objectifs du niveau ont été atteints.

Returns :

L'état de victoire actuel.

Return type

bool

compare_truth_tables()

Valide la puce actuelle par rapport à la table de vérité de la solution attendue.

Returns :

True si les tables de vérité correspondent, False sinon.

Return type

bool

get_gates(*chip=None*)

Identifie tous les identifiants des portes logiques fonctionnelles.

Args :

chip : La puce à inspecter.

Returns :

Une liste d'identifiants de portes pour les portes standards ou personnalisées.

Parameters

chip (Chip / None)

Return type

List[str]

get_inputs(*chip=None*)

Identifie tous les identifiants des portes d'entrée.

Args :

chip : La puce à inspecter.

Returns :

Une liste d'identifiants de portes agissant comme entrées.

Parameters**chip** (*Chip* / *None*)**Return type***List*[*str*]**get_outputs**(*chip=None*)

Identifie tous les identifiants des portes de sortie.

Args :

chip : La puce à inspecter.

Returns :

Une liste d'identifiants de portes agissant comme sorties.

Parameters**chip** (*Chip* / *None*)**Return type***List*[*str*]**get_single_truth_table_complex**(*chip*)

Génère une table de vérité pour la puce complexe fournie.

Args :

chip : L'instance à évaluer.

Parameters**chip** (*Chip*)**Return type***None***get_single_truth_table_simple**(*chip*)

Génère une table de vérité pour la puce simple fournie.

Args :

chip : L'instance à évaluer.

Parameters**chip** (*Chip*)**Return type***None***get_stars_count**()

Calcule le nombre d'étoiles actuel en fonction du temps et de l'utilisation des indices.

Returns :

Le nombre d'étoiles obtenues, allant de 0 à 3.

Return type*int*

get_truth_table(*answer=False*)

Initialise le stockage de la table de vérité et déclenche la génération.

Args :

answer : Si True, évalue la puce solution ; sinon, la puce du joueur.

Parameters

answer (*bool*)

Return type

None

load(*data*)

Alimente l'état du niveau à partir d'un dictionnaire de configuration.

Args :

data : Le dictionnaire source contenant les paramètres du niveau.

Parameters

data (*Dict[str, Any]*)

Return type

None

play_mode()

Prépare le niveau pour le jeu en réinitialisant l'état et en définissant la logique cible.

Return type

None

save()

Séréalise le niveau actuel et la configuration de la puce dans un fichier.

Return type

None

start_chip(*chip=None*)

Réinitialise les états d'E/S (Entrées/Sorties) des portes au sein de la puce spécifiée.

Args :

chip : La puce à initialiser ; utilise la puce actuelle par défaut si None.

Parameters

chip (*Chip / None*)

Return type

None

class modules.data.loader.**Loader**

Gère le chargement des ressources JSON externes, des ressources de logique de jeu, ainsi que la génération des textures procédurales de l'interface utilisateur.

bake_custom_gates()

Génère les textures pour les puces personnalisées définies par l'utilisateur.

Return type

None

bake_level_buttons()

Génère des boutons visuels uniques pour chaque niveau de jeu disponible.

Return type

None

bake_predefined_gates()

Génère les textures pour toutes les portes système intégrées.

Return type

None

bake_single_gate(*gate*, *id*)

Met en cache les textures spécifiques aux états pour une porte afin d'optimiser les performances.

Args:

gate: L'instance de la porte. *id*: Identifiant unique de la porte.

Returns:

True si la génération a réussi, False si la complexité des entrées/sorties dépasse les limites.

Parameters

- **gate** (*Any*)
- **id** (*str*)

Return type

bool

bake_textures()

Coordonne le flux complet de génération procédurale de textures.

Return type

None

load()

Exécute le pipeline complet de chargement des ressources et d'initialisation.

Return type

None

load_files(*sub_folder*)

Lit tous les fichiers JSON d'un sous-répertoire spécifié.

Args:

sub_folder: Le chemin du répertoire relatif à la racine des données.

Returns:

Une liste contenant le contenu analysé de chaque fichier JSON.

Parameters

sub_folder (*str*)

Return type

List[Dict[str, Any]]

load_fonts()

Enregistre les polices TTF externes auprès du moteur de rendu.

Return type

None

load_levels()

Charge toutes les définitions de niveaux et leurs ressources requises.

Return type

None

load_saves()

Charge et traite toutes les configurations de puces sauvegardées.

Return type

None

load_saves_dependency()

Résout itérativement les dépendances jusqu'à ce que toutes les puces soient chargées ou que la limite soit atteinte.

Return type

None

load_saves_dependency_round()

Effectue une seule passe de résolution sur les puces en tampon.

Return type

None

load_single_chip(*raw_data*)

Enregistre un objet "chip" et gère ses exigences en matière de dépendances.

Args:

raw_data: Le dictionnaire de configuration pour la puce.

Parameters

raw_data (*Dict[str, Any]*)

Return type

None

load_tilesets()

Découpe les planches de sprites en grilles de textures pour les composants de l'interface.

Return type

None

load_ui()

Charge et initialise les ressources statiques de l'interface.

Return type

None

render_gate_image(*gate*)

Génère une image composite pour une porte logique, incluant les tuiles et les étiquettes de texte.

Args:

gate: Le composant de porte à restituer.

Returns:

L'image PIL générée.

Parameters**gate** (*CustomGate* / *Any*)**Return type***Image*

Fournit la classe de base `Node` pour les éléments de la boîte à outils de l'interface utilisateur.

class `modules.data.node.Node(id)`

Représente un élément structurel au sein du système de boîte à outils de l'interface utilisateur.

Parameters**id** (*Any*)**draw()**

Affiche le nœud dans l'interface utilisateur.

Return type

None

8.1.1 Data Nodes

Assure la gestion des nœuds de chemin et le rendu géométrique des tracés pour l'éditeur d'interface utilisateur.

class `modules.data.nodes.path.Path(id)`

Gère les segments de ligne ramifiés et leurs boîtes de collision (hitboxes) associées.

Parameters**id** (*Any*)**abort()**

Réinitialise l'état du chemin en cours d'attente.

Return type

None

add_path()

Ajoute un point ou un segment à la branche active actuelle.

Return type

None

clean_out_single_branch(*depth=0*)

Supprime de manière récursive les branches ayant des connexions insuffisantes.

Args:

depth: Profondeur actuelle de récursion utilisée pour interrompre les cycles.

Parameters**depth** (*int*)**Return type**

None

draw()

Affiche les lignes du chemin, les points actifs et les boîtes de collision.

Return type

None

property empty: bool

Détermine si le chemin ne possède aucune géométrie significative.

finish()

Valide le segment de chemin actuel et prépare une nouvelle branche.

Return type

None

generate_thick_line_polygon(*points*, *thickness*)

Construit un polygone représentant un segment de ligne épaisse.

Args:

points: Coordonnées formant l'épine dorsale du segment. *thickness*: Largeur totale du polygone.

Returns:

Tuple des listes de sommets des bords gauche et droit du polygone.

Parameters

- **points** (*List[Tuple[float, float]]*)
- **thickness** (*float*)

Return type

Tuple[List[Tuple[float, float]], List[Tuple[float, float]]]

get_connected_points(*target_id*)

Récupère les connexions associées à un ID de nœud spécifique.

Args:

target_id: Identifiant du nœud cible.

Returns:

Tuple contenant les listes des entrées et sorties connectées.

Parameters

target_id (*Any*)

Return type

Tuple[List[List[Any]], List[List[Any]]]

property get_touched_branch: int | None

Identifie l'index de la branche intersectant actuellement le curseur.

load(*data*)

Restaure la configuration du chemin à partir d'un état sauvegardé.

Args:

data: Dictionnaire contenant les propriétés sérialisées du chemin.

Parameters

data (*Dict[str, Any]*)

Return type

None

load_hitboxes(*hitboxes*)

Reconstruit les objets de boîte de collision à partir des données sauvegardées.

Args:

hitboxes: Dictionnaire des structures de boîtes de collision sauvegardées.

Parameters

hitboxes (*Dict[str, List[Dict[str, Any]]]*)

Return type

Dict[int, List[PolyHitbox]]

merge(*path*)

Intègre les données d'un chemin externe dans l'instance locale.

Args:

path: Objet Path cible à fusionner.

Parameters

path (*Path*)

Return type

None

project_point_onto_segments(*x, y*)

Calcule le point le plus proche sur n'importe quel segment par rapport aux coordonnées données.

Args:

x: Coordonnée cible. y: Coordonnée cible.

Returns:

Dictionnaire contenant le point le plus proche, l'ID de la branche, l'index et la distance.

Parameters

- **x** (*float*)
- **y** (*float*)

Return type

Dict[str, Any]

recalculate_hitbox()

Régénère la géométrie de collision pour toutes les branches en fonction du décalage actuel de la caméra.

Return type

None

remove_branch(*branch*)

Supprime une branche et recalibre les indices restants.

Args:

branch: Index cible de la branche à supprimer.

Parameters

branch (*int*)

Return type

None

save()

Sérialise l'état complet du Path pour la persistance des données.

Return type

Dict[str, Any]

save_hitboxes()

Sérialise l'état des boîtes de collision dans une structure de dictionnaire.

Return type

Dict[int, List[Dict[str, Any]]]

property touched: bool

Indique si l'une des boîtes de collision des segments est active.

Eight-Bit Nodes Fournit des implémentations de portes logiques 8 bits pour la simulation de circuits.

class modules.data.nodes.eight.addr.**Adder**(*id*)

Représente un composant de porte logique de type additionneur complet (Full Adder) 8 bits.

Attributes:

name: Le nom d'affichage de la porte. gate_type: L'identifiant de la catégorie de la porte logique. inputs: Décalages (offsets) des broches pour les signaux d'entrée. outputs: Décalages (offsets) des broches pour les signaux de sortie. inputs_sizes: Largeur en bits des broches d'entrée. outputs_sizes: Largeur en bits des broches de sortie.

Parameters

id (*int* | *str*)

class modules.data.nodes.eight.gand.**And**(*id*)

Représente un composant de porte logique ET (AND) 8 bits.

Attributes :

name : Le nom d'affichage de la porte. gate_type : L'identifiant de la catégorie de la porte logique. inputs : Décalages des broches (pin offsets) pour les signaux d'entrée. outputs : Décalages des broches (pin offsets) pour les signaux de sortie. inputs_sizes : Largeur de bits des broches d'entrée. outputs_sizes : Largeur de bits des broches de sortie.

Parameters

id (*Any*)

class modules.data.nodes.eight.gnot.**Not**(*id*)

Représente un composant de porte logique NON (NOT) 8 bits.

Attributes :

name : Le nom d'affichage de la porte. gate_type : L'identifiant de la catégorie de porte logique. inputs : Décalages de broches (offsets) pour les signaux d'entrée. outputs : Décalages de broches (offsets) pour les signaux de sortie. inputs_sizes : Largeur de bit des broches d'entrée. outputs_sizes : Largeur de bit des broches de sortie.

Parameters

id (*Any*)

class modules.data.nodes.eight.gor.**Or**(*id*)

Représente un composant de porte logique OU (OR) 8 bits.

Attributs:

name: Le nom d’affichage de la porte. gate_type: L’identifiant de la catégorie de porte logique. inputs: Décalages (offsets) des broches pour les signaux d’entrée. outputs: Décalages (offsets) des broches pour les signaux de sortie. inputs_sizes: Largeur de bits des broches d’entrée. outputs_sizes: Largeur de bits des broches de sortie.

Parameters

id (*Any*)

Fournit des composants de portes logiques spécialisés pour les environnements de simulation.

class modules.data.nodes.eight.input.**Input**(*id*)

Représente une porte d’entrée 8 bits au sein d’un circuit de simulation logique.

Parameters

id (*Any*)

switch()

Simule un changement d’état d’entrée en générant un nouvel entier 8 bits aléatoire.

Return type

None

class modules.data.nodes.eight.nand.**Nand**(*id*)

Représente un composant de porte logique Nand 8-bit.

Attributs :

name : Le nom d’affichage de la porte. gate_type : L’identifiant de la catégorie de la porte logique. inputs : Décalages des broches pour les signaux d’entrée. outputs : Décalages des broches pour les signaux de sortie. inputs_sizes : Largeur de bits des broches d’entrée. outputs_sizes : Largeur de bits des broches de sortie.

Parameters

id (*Any*)

class modules.data.nodes.eight.nor.**Nor**(*id*)

Représente un composant de porte logique NOR 8 bits.

Attributs :

name : Le nom d’affichage de la porte. gate_type : L’identifiant de la catégorie de porte logique. inputs : Décalages des broches pour les signaux d’entrée. outputs : Décalages des broches pour les signaux de sortie. inputs_sizes : Largeur en bits des broches d’entrée. outputs_sizes : Largeur en bits des broches de sortie.

Parameters

id (*Any*)

class modules.data.nodes.eight.one.**ONE**(*id*)

Représente une valeur logique 1.

Attributs :

name : Le nom d’affichage de la porte. gate_type : L’identifiant de la catégorie de porte logique. inputs : Décalages de broches (offsets) pour les signaux d’entrée. outputs : Décalages de broches (offsets) pour les signaux de sortie. inputs_sizes : Largeur de bits des broches d’entrée. outputs_sizes : Largeur de bits des broches de sortie.

Parameters**id** (*Any*)**switch()**

Bascule l'état du composant.

class modules.data.nodes.eight.output.**Output**(*id*)

Représente un composant de porte de sortie dans le système d'interface utilisateur.

Parameters**id** (*Any*)

Fournit des implémentations de portes logiques 8 bits pour la simulation de circuits.

class modules.data.nodes.eight.sub.**Sub**(*id*)

Représente un composant de porte logique de soustraction complète (Full Sub) 8 bits.

Attributes :

name : Le nom d'affichage de la porte. gate_type : L'identifiant de la catégorie de porte logique. inputs : Décalages des broches pour les signaux d'entrée. outputs : Décalages des broches pour les signaux de sortie. inputs_sizes : Largeur de bits des broches d'entrée. outputs_sizes : Largeur de bits des broches de sortie.

Parameters**id** (*int* / *str*)**class** modules.data.nodes.eight.xor.**Xor**(*id*)

Représente un composant de porte logique XOR 8 bits.

Attributes :

name : Le nom d'affichage de la porte. gate_type : L'identifiant de la catégorie de la porte logique. inputs : Décalages (offsets) des broches pour les signaux d'entrée. outputs : Décalages (offsets) des broches pour les signaux de sortie. inputs_sizes : Largeur de bits des broches d'entrée. outputs_sizes : Largeur de bits des broches de sortie.

Parameters**id** (*Any*)**Mix Nodes****class** modules.data.nodes.mix.eight_breaker.**Breaker**(*id*)

Représente un composant de porte de rupture 8-bit pour l'acheminement des données système.

Parameters**id** (*Any*)

Fournit l'implémentation de la porte Maker pour l'agrégation de signaux 8 bits.

class modules.data.nodes.mix.eight_maker.**Maker**(*id*)

Agrège huit entrées de signaux 1 bit en une seule sortie de 8 bits.

Parameters**id** (*Any*)**One-Bit Nodes** Fournit les implémentations pour les composants de portes logiques standard.


```
class modules.data.nodes.one.addr.Adder(id)
```

Représente une porte additionneur complet (Full Adder) standard.

Parameters

id (*int* / *str*)

```
class modules.data.nodes.one.clock.Clock(id)
```

Représente un générateur de signal d'horloge au sein de l'environnement de simulation.

Parameters

id (*int* / *str*)

Fournit l'implémentation de la porte de retard (Delay) pour les simulations de circuits logiques.

```
class modules.data.nodes.one.delay.Delay(id)
```

Porte de retard, maintient le signal pendant 1 cycle (tick).

Parameters

id (*Any*)

Fournit les implémentations pour les composants de portes logiques standard.

```
class modules.data.nodes.one.gand.And(id)
```

Représente une porte logique ET standard à deux entrées.

Parameters

id (*int* / *str*)

```
class modules.data.nodes.one.gnot.Not(id)
```

Représente une porte logique NON.

Parameters

id (*Any*)

```
class modules.data.nodes.one.gor.Or(id)
```

Représente un composant de porte logique OU au sein de la simulation.

Parameters

id (*str*)

Fournit l'implémentation de la porte Pass pour les simulations de circuits logiques.

```
class modules.data.nodes.one.gpass.Pass(id)
```

Composant tampon (buffer) qui propage les signaux d'entrée sans modification.

Parameters

id (*Any*)

```
class modules.data.nodes.one.input.Input(id)
```

Représente une porte de commutation d'entrée manuelle au sein d'un circuit.

Parameters

id (*Any*)

switch()

Bascule l'état de sortie actuel entre 0 et 1.

Return type

None

Fournit des implémentations pour les portes logiques standard utilisées dans la simulation de circuits.

```
class modules.data.nodes.one.nand.Nand(id)
```

Représente une porte logique NON-ET (NAND) dans une simulation de circuit.

Parameters

id (*int* / *str*)

```
class modules.data.nodes.one.nor.Nor(id)
```

Représente une porte logique NON-OU (NOR) avec deux entrées et une sortie.

Parameters

id (*Any*)

```
class modules.data.nodes.one.off.Off(id)
```

Représente une porte logique OFF.

Parameters

id (*Any*)

```
class modules.data.nodes.one.on.On(id)
```

Représente une porte logique de type ON.

Parameters

id (*Any*)

```
class modules.data.nodes.one.output.Output(id)
```

Représente un nœud de sortie dans le circuit.

Attributs :

name : Nom d'identifiant interne pour la porte. type : Classification générale de la porte. gate_type : Type opérationnel spécifique. inputs : Liste des états d'entrée booléens actuels. outputs : Liste des terminaux de sortie connectés. inputs_sizes : Dimensions requises pour les ports d'entrée. outputs_sizes : Dimensions requises pour les ports de sortie. exceptional_size_offset : Décalage vertical pour la logique de rendu. gate_width : Envergure horizontale de la représentation de la porte. gate_tile_pattern : Plan de grille aplati pour le rendu graphique.

Parameters

id (*Any*)

Fournit des implémentations pour les composants de portes logiques standards.

```
class modules.data.nodes.one.sub.Sub(id)
```

Représente une porte logique standard de type Soustracteur Complet (Full Sub).

Parameters

id (*int* / *str*)

Fournit l'implémentation de la porte SWC pour les simulations de circuits logiques.

```
class modules.data.nodes.one.swc.Swc(id)
```

Porte Swc, offrant un moyen d'interrompre la sortie d'une porte, empêchant sa propagation ultérieure.

Parameters

id (*Any*)

Fournit les implémentations pour les composants de portes logiques standard.

```
class modules.data.nodes.one.tand.TAnd(id)
```

Représente une porte logique ET standard à deux entrées.

Parameters

id (*int* / *str*)

```
class modules.data.nodes.one.tor.Tor(id)
```

Représente un composant de porte logique OU au sein de la simulation.

Parameters

id (*str*)

```
class modules.data.nodes.one.xor.Xor(id)
```

Représente un composant de porte logique XOR.

Parameters

id (*Any*)

8.2 Engine Module

Ce module gère l'intégration de la logique principale du moteur de jeu. Il agit comme une interface pour les composants essentiels du système.

```
class modules.engine.logic.Engine
```

Simule le comportement des puces de circuit et la propagation des signaux.

```
calculate_custom(gate)
```

Simule la logique interne pour un composant de porte personnalisé.

Args:

gate: Objet de porte personnalisée à simuler.

Parameters

gate (*CustomGate*)

Return type

None

```
calculate_output(gate_type, inputs, gate)
```

Calcule la sortie pour une porte logique standard.

Args:

gate_type: Identifiant pour la logique de la porte. *inputs*: Valeurs d'entrée pour la porte.

Returns:

Valeurs de sortie sous forme de liste.

Parameters

- **gate_type** (*str*)
- **inputs** (*list[int]*)
- **gate** (*Gate*)

Return type

list[int]

```
draw_connections(chip, inputs, outputs, gates)
```

Construit une carte de connectivité pour la puce.

Args:

chip: La structure de la puce. *inputs*: Liste des IDs des portes d'entrée. *outputs*: Liste des IDs des portes de sortie. *gates*: Liste des IDs des portes logiques.

Returns:

Dictionnaire associant les IDs de portes aux structures de chemin de connexion.

Parameters

- **chip** ([Chip](#))
- **inputs** (*list[str]*)
- **outputs** (*list[str]*)
- **gates** (*list[str]*)

Return type

dict[str, list]

get_wired_inputs_map(*connections*)

Associe les ports d'entrée des portes à leur état de câblage.

Args:

connections: Carte de connectivité de la puce.

Returns:

Dictionnaire associant l'ID de la porte à un ensemble d'indices de ports d'entrée actifs.

Parameters

connections (*dict[str, list]*)

Return type

dict[str, set[int]]

propagate_outputs(*chip, connections, source_id, visible=False*)

Propage le signal d'une porte source vers les cibles connectées.

Args:

chip: La structure de la puce. connections: Carte de connectivité de la puce. source_id: ID de la porte à l'origine du signal.

Parameters

- **chip** ([Chip](#))
- **connections** (*dict[str, list]*)
- **source_id** (*str*)

Return type

None

propagate_values(*chip, visible=True*)

Initialise et exécute la séquence de propagation du signal.

Args:

chip: L'objet puce à simuler.

Parameters

chip ([Chip](#))

Return type

None

reset_input_validation(*chip, gates, outputs*)

Réinitialise l'état de validation pour toutes les portes et tous les chemins.

Args:

chip: L'objet puce à réinitialiser. gates: IDs des portes logiques. outputs: IDs des portes de sortie.

Parameters

- **chip** ([Chip](#))
- **gates** (*list[str]*)
- **outputs** (*list[str]*)

Return type

None

run_propagation_loop(*chip, connections, gates, inputs, outputs, visible=False*)

Simule de manière itérative la propagation du signal à travers les portes logiques.

Args:

chip: La puce en cours de simulation. connections: Carte de connectivité. gates: IDs des portes logiques. inputs: IDs des portes d'entrée. outputs: IDs des portes de sortie.

Parameters

- **chip** ([Chip](#))
- **connections** (*dict[str, list]*)
- **gates** (*list[str]*)
- **inputs** (*list[str]*)
- **outputs** (*list[str]*)
- **visible** (*bool*)

Return type

None

sort_gates(*chip*)

Catégorise les composants de la puce en portes logiques, d'entrée et de sortie.

Args:

chip: La structure de la puce à analyser.

Returns:

Tuple contenant les listes d'IDs des portes logiques, des entrées et des sorties.

Parameters

- **chip** ([Chip](#))

Return type

tuple[list[str], list[str], list[str]]

modules.engine.logic.gate_8adder(*inputs*)

Additionneur 8 bits sans sortie de retenue.

Args:

inputs: [A, B] où $0 \leq A, B \leq 255$

Returns:

Résultat 8 bits (0–255)

Parameters

inputs (*list[int]*)

Return type

int

`modules.engine.logic.gate_8and(inputs)`

Effectue un ET (AND) sur 8 bits.

Args:

inputs: Liste contenant deux valeurs entières de 8 bits.

Returns:

ET bit à bit des entrées.

Parameters

inputs (*list[int]*)

Return type

list[int]

`modules.engine.logic.gate_8breaker(inputs)`

Décompose un entier de 8 bits en une séquence de bits.

Args:

inputs: Liste contenant l'entier à décomposer.

Returns:

Représentation binaire 8 bits sous forme de liste d'entiers.

Parameters

inputs (*list[int]*)

Return type

list[int]

`modules.engine.logic.gate_8maker(inputs)`

Convertit une séquence de bits en un entier de 8 bits.

Args:

inputs: Liste de bits individuels.

Returns:

Liste à un seul élément contenant la représentation entière.

Parameters

inputs (*list[int]*)

Return type

list[int]

`modules.engine.logic.gate_8mux(inputs)`

Utilise la première entrée comme sélecteur pour choisir l'une des 8 entrées suivantes.

Args:

inputs: Une liste où l'index 0 est le sélecteur (0-7)

et les indices 1-8 sont les entrées de données.

Returns:

La valeur d'entrée choisie.

Parameters

inputs (*list[int]*)

Return type

list[int]

`modules.engine.logic.gate_8nand(inputs)`

Effectue un NON-ET (NAND) sur 8 bits.

Args:

inputs: Liste contenant deux valeurs entières de 8 bits.

Returns:

NAND bit à bit des entrées.

Parameters

inputs (*list[int]*)

Return type

list[int]

`modules.engine.logic.gate_8nor(inputs)`

Effectue un NON-OU (NOR) sur 8 bits.

Args:

inputs: Liste contenant deux valeurs entières de 8 bits.

Returns:

NOR bit à bit des entrées.

Parameters

inputs (*list[int]*)

Return type

list[int]

`modules.engine.logic.gate_8not(inputs)`

Effectue une inversion sur 8 bits via XOR.

Args:

inputs: Liste contenant une valeur entière de 8 bits.

Returns:

Valeur 8 bits inversée bit à bit.

Parameters

inputs (*list[int]*)

Return type

list[int]

`modules.engine.logic.gate_8or(inputs)`

Effectue un OU (OR) sur 8 bits.

Args:

inputs: Liste contenant deux valeurs entières de 8 bits.

Returns:

OU bit à bit des entrées.

Parameters

inputs (*list[int]*)

Return type

list[int]

`modules.engine.logic.gate_8subtractor(inputs)`

Soustracteur 8 bits sans retenue empruntée.

Args:

inputs: [A, B] où $0 \leq A, B \leq 255$

Returns:

Résultat 8 bits (0–255)

Parameters

inputs (*list[int]*)

Return type

int

`modules.engine.logic.gate_8xor(inputs)`

Effectue un OU exclusif (XOR) sur 8 bits.

Args:

inputs: Liste contenant deux valeurs entières de 8 bits.

Returns:

XOR bit à bit des entrées.

Parameters

inputs (*list[int]*)

Return type

list[int]

`modules.engine.logic.gate_adder(inputs)`

Calcule l'opération d'un additionneur complet (Full Adder).

Args:

inputs: Liste contenant trois bits d'entrée [A, B, Cin].

Returns:

[Somme, Retenue sortante]

Parameters

inputs (*list[int]*)

Return type

list[int]

`modules.engine.logic.gate_and(inputs)`

Calcule l'opération ET (AND).

Args:

inputs: Liste contenant deux bits d'entrée.

Returns:

Résultat de l'opération ET.

Parameters

inputs (*list[int]*)

Return type

list[int]

`modules.engine.logic.gate_clk(inputs=[])`

Fournit un signal d'horloge basé sur le temps système.

Args:

inputs: Liste d'entrées ignorée.

Returns:

Bit alternant basé sur l'horodatage Unix actuel.

Parameters

inputs (*list[int]*)

Return type

list[int]

`modules.engine.logic.gate_delay(inputs, gate)`

Maintient le signal pendant 1 tick.

Args:

inputs: Liste contenant un bit d'entrée.

Returns:

Bit tel qu'il était il y a un tick.

Parameters

- **inputs** (*list[int]*)
- **gate** (*Gate*)

Return type

list[int]

`modules.engine.logic.gate_nand(inputs)`

Calcule l'opération NON-ET (NAND).

Args:

inputs: Liste contenant deux bits d'entrée.

Returns:

Résultat de l'opération NAND.

Parameters

inputs (*list[int]*)

Return type

list[int]

`modules.engine.logic.gate_nor(inputs)`

Calcule l'opération NON-OU (NOR).

Args:

inputs: Liste contenant deux bits d'entrée.

Returns:

Résultat de l'opération NOR.

Parameters

inputs (*list[int]*)

Return type

list[int]

`modules.engine.logic.gate_not(inputs)`

Calcule l'opération NON (NOT).

Args:

inputs: Liste contenant un seul bit d'entrée.

Returns:

Bit inversé.

Parameters

inputs (*list[int]*)

Return type

list[int]

`modules.engine.logic.gate_or(inputs)`

Calcule l'opération OU (OR).

Args:

inputs: Liste contenant deux bits d'entrée.

Returns:

Résultat de l'opération OU.

Parameters

inputs (*list[int]*)

Return type

list[int]

`modules.engine.logic.gate_pass(inputs)`

Transmet les valeurs d'entrée sans modification.

Args:

inputs: Liste d'entrée à transmettre.

Returns:

Liste d'entrée inchangée.

Parameters

inputs (*list[int]*)

Return type

list[int]

`modules.engine.logic.gate_register(inputs, gate)`

Stocke une valeur unique de 8 bits.

Args:

inputs: Valeur d'entrée (8), Sauvegarder (Save), Charger (Load)

Returns:

0 ou la valeur stockée.

Parameters

- **inputs** (*list[int]*)
- **gate** (*Gate*)

Return type

list[int]

`modules.engine.logic.gate_subtractor(inputs)`

Calcule l'opération d'un soustracteur complet (Full Subtractor).

Args:

inputs: Liste contenant trois bits d'entrée [A, B, Bin].

Returns:

[Différence, Retenue empruntée]

Parameters**inputs** (*list[int]*)**Return type**

list[int]

`modules.engine.logic.gate_tand(inputs)`

Calcule l'opération ET (AND).

Args:

inputs: Liste contenant trois bits d'entrée.

Returns:

Résultat de l'opération ET.

Parameters**inputs** (*list[int]*)**Return type**

list[int]

`modules.engine.logic.gate_tor(inputs)`

Calcule l'opération OU (OR).

Args:

inputs: Liste contenant trois bits d'entrée.

Returns:

Résultat de l'opération OU.

Parameters**inputs** (*list[int]*)**Return type***list[int]*`modules.engine.logic.gate_xor(inputs)`

Calcule l'opération OU exclusif (XOR).

Args:

inputs: Liste contenant deux bits d'entrée.

Returns:

Résultat de l'opération XOR.

Parameters**inputs** (*list[int]*)**Return type***list[int]*

8.3 Logger Module

`class modules.logger.Logger(name)`

Gère la journalisation formatée dans la console pour les processus de l'application.

Attributs:

name: L'identifiant du processus utilisant cette instance. levels: Étiquettes de classification de la sévérité. colors: Séquences d'échappement ANSI pour le style de sortie du terminal. history: Entrées de journal capturées au cours de la session actuelle.

Parameters**name** (*str*)`debug(message)`

Enregistre un message avec la sévérité DEBUG.

Args:

message: Le contenu à journaliser.

Parameters**message** (*str*)**Return type**

None

`error(message)`

Enregistre un message avec la sévérité ERROR.

Args:

message: Le contenu à journaliser.

Parameters**message** (*str*)**Return type**

None

info(*message*)

Enregistre un message avec la sévérité INFO.

Args:

message: Le contenu à journaliser.

Parameters

message (*str*)

Return type

None

print(*message*)

Enregistre un message avec la sévérité INFO par défaut (alias).

Args:

message: Le contenu à journaliser.

Parameters

message (*str*)

Return type

None

success(*message*)

Enregistre un message avec la sévérité SUCCESS.

Args:

message: Le contenu à journaliser.

Parameters

message (*str*)

Return type

None

warning(*message*)

Enregistre un message avec la sévérité WARNING.

Args:

message: Le contenu à journaliser.

Parameters

message (*str*)

Return type

None

8.4 UI Module

Fournit des capacités de suivi de la souris avec alignement sur la grille et analyse directionnelle. Fournit la gestion des fenêtres et l'état de navigation des vues pour l'application LogicBox.

class modules.ui.window.Window

Gère la fenêtre principale de l'application et fournit un système de navigation basé sur une pile.

back()

Navigue vers la vue précédente dans la pile d'historique.

Return type

None

display(*view*)

Ajoute une nouvelle vue à la pile d'historique et l'affiche.

Args:

view: L'instance de la vue à afficher.

Parameters

view (*View*)

Return type

None

first()

Réinitialise la navigation vers la vue initiale et vide la pile d'historique.

Return type

None

hide()

Masque la vue actuellement active.

Return type

None

run()

Lance la boucle d'événements principale de l'application.

Return type

None

8.4.1 UI Components

Debug Display .. automodule:: modules.ui.debug_display_all_tiles

members

class modules.ui.debug_display_all_tiles.view.**DebugTilesView**

Gère la logique de visualisation et de sélection pour plusieurs jeux de tuiles d'atouts.

load_tilesets()

Analyse les chemins des jeux de tuiles et extrait les grilles de textures pour chaque entrée.

Return type

None

on_draw()

Affiche le jeu de tuiles actif, l'interface de la grille et les étiquettes de retour utilisateur.

Return type

None

on_key_press(*key*, *key_modifiers*)

Traite les touches de navigation pour la sélection des jeux de tuiles et le contrôle de l'application.

Args:

key: Le code de la touche déclenchée. *key_modifiers*: Masque de bits des touches de modification actives.

Parameters

- **key** (*int*)
- **key_modifiers** (*int*)

Return type

None

on_key_release(*key*, *key_modifiers*)

Structure pour les événements de relâchement de touche du clavier.

Parameters

- **key** (*int*)
- **key_modifiers** (*int*)

Return type

None

on_mouse_motion(*x*, *y*, *delta_x*, *delta_y*)

Met à jour `hovered_index` en fonction des coordonnées actuelles de la souris.

Args:

x: Coordonnée horizontale de la souris sur l'écran. *y*: Coordonnée verticale de la souris sur l'écran. *delta_x*: Variation de la position horizontale de la souris. *delta_y*: Variation de la position verticale de la souris.

Parameters

- **x** (*float*)
- **y** (*float*)
- **delta_x** (*float*)
- **delta_y** (*float*)

Return type

None

on_mouse_press(*x*, *y*, *button*, *key_modifiers*)

Structure pour les événements de pression des boutons de la souris.

Parameters

- **x** (*float*)
- **y** (*float*)
- **button** (*int*)
- **key_modifiers** (*int*)

Return type

None

on_mouse_release(*x, y, button, key_modifiers*)

Structure pour les événements de relâchement des boutons de la souris.

Parameters

- **x** (*float*)
- **y** (*float*)
- **button** (*int*)
- **key_modifiers** (*int*)

Return type

None

on_update(*delta_time*)

Structure pour la logique de mise à jour spécifique à chaque image (frame).

Parameters

- **delta_time** (*float*)

Return type

None

Editor .. automodule:: modules.ui.editor

members

Fournit la vue InputValue pour l'édition et la sauvegarde des données de configuration des portes.

class modules.ui.editor.input.**InputFrame**(*chip, gate*)

Gère la disposition de l'interface utilisateur et les interactions pour l'édition des propriétés de porte.

Parameters

- **chip** (*Any*)
- **gate** (*str*)

on_draw()

Affiche tous les éléments textuels configurés de l'interface utilisateur.

Return type

None

on_key_press(*key, key_modifiers*)

Gère les événements de pression de touches du clavier.

Args:

key: L'identifiant de la touche pressée. key_modifiers: Drapeaux binaires pour les touches de modification.

Parameters

- **key** (*int*)
- **key_modifiers** (*int*)

Return type

None

on_key_release(*key*, *key_modifiers*)

Gère les événements de relâchement de touches.

Parameters

- **key** (*int*)
- **key_modifiers** (*int*)

Return type

None

on_mouse_motion(*x*, *y*, *delta_x*, *delta_y*)

Met à jour l'état global du suivi de la souris.

Args:

x: La coordonnée *x* actuelle de la souris. *y*: La coordonnée *y* actuelle de la souris. *delta_x*: La variation de la coordonnée *x* depuis la dernière image. *delta_y*: La variation de la coordonnée *y* depuis la dernière image.

Parameters

- **x** (*float*)
- **y** (*float*)
- **delta_x** (*float*)
- **delta_y** (*float*)

Return type

None

on_mouse_press(*x*, *y*, *button*, *key_modifiers*)

Traite les interactions de clic de souris avec les éléments de l'interface.

Args:

x: La coordonnée *x* du clic de souris. *y*: La coordonnée *y* du clic de souris. *button*: Le bouton de la souris pressé. *key_modifiers*: Drapeaux binaires pour les touches de modification.

Parameters

- **x** (*float*)
- **y** (*float*)
- **button** (*int*)
- **key_modifiers** (*int*)

Return type

None

on_mouse_release(*x*, *y*, *button*, *key_modifiers*)

Gère les événements de relâchement de bouton de souris.

Parameters

- **x** (*float*)
- **y** (*float*)
- **button** (*int*)

- **key_modifiers** (*int*)

Return type

None

on_update(*delta_time*)

Gère les mises à jour périodiques de la logique.

Parameters**delta_time** (*float*)**Return type**

None

reset()

Réinitialise l'état interne de la vue.

Return type

None

Fournit la vue SaveFrame pour l'édition et la sauvegarde des données de configuration des puces.

class modules.ui.editor.save.**SaveFrame**(*chip*)

Gère la disposition de l'interface utilisateur et les interactions pour l'édition des propriétés de la puce.

Parameters**chip** (*Any*)**on_draw**()

Rendu de tous les éléments textuels configurés de l'interface utilisateur.

Return type

None

on_key_press(*key*, *key_modifiers*)

Gère les événements de pression de touches du clavier.

Args:

key: L'identifiant de la touche pressée. key_modifiers: Drapeaux binaires pour les touches de modification.

Parameters

- **key** (*int*)
- **key_modifiers** (*int*)

Return type

None

on_key_release(*key*, *key_modifiers*)

Gère les événements de relâchement de touches.

Parameters

- **key** (*int*)
- **key_modifiers** (*int*)

Return type

None

on_mouse_motion(*x*, *y*, *delta_x*, *delta_y*)

Met à jour l'état global du suivi de la souris.

Args:

x: La coordonnée x actuelle de la souris. *y*: La coordonnée y actuelle de la souris. *delta_x*: La variation de la coordonnée x depuis la dernière image. *delta_y*: La variation de la coordonnée y depuis la dernière image.

Parameters

- ***x*** (*float*)
- ***y*** (*float*)
- ***delta_x*** (*float*)
- ***delta_y*** (*float*)

Return type

None

on_mouse_press(*x*, *y*, *button*, *key_modifiers*)

Traite les interactions par clic de souris avec les éléments de l'interface.

Args:

x: La coordonnée x du clic de souris. *y*: La coordonnée y du clic de souris. *button*: Le bouton de la souris pressé. *key_modifiers*: Drapeaux binaires pour les touches de modification.

Parameters

- ***x*** (*float*)
- ***y*** (*float*)
- ***button*** (*int*)
- ***key_modifiers*** (*int*)

Return type

None

on_mouse_release(*x*, *y*, *button*, *key_modifiers*)

Gère les événements de relâchement du bouton de la souris.

Parameters

- ***x*** (*float*)
- ***y*** (*float*)
- ***button*** (*int*)
- ***key_modifiers*** (*int*)

Return type

None

on_update(*delta_time*)

Gère les mises à jour périodiques de la logique.

Parameters

- delta_time*** (*float*)

Return type

None

reset()

Réinitialise l'état interne de la vue.

Return type

None

class modules.ui.editor.selector.EditorChipSelector

Fournit un menu de sélection pour les puces existantes et la navigation vers l'éditeur.

on_draw()

Rendu des éléments de l'interface et de leurs zones de collision (hitboxes) associées.

Return type

None

on_key_press(*key*, *key_modifiers*)

Gère les entrées clavier.

Args:

key: Le code de la touche pressée. *key_modifiers*: Modificateurs de touches actifs (opérations bit à bit).

Parameters

- **key** (*int*)
- **key_modifiers** (*int*)

Return type

None

on_key_release(*key*, *key_modifiers*)

Gère les événements de relâchement de touches.

Args:

key: Le code de la touche relâchée. *key_modifiers*: Modificateurs de touches actifs (opérations bit à bit).

Parameters

- **key** (*int*)
- **key_modifiers** (*int*)

Return type

None

on_mouse_motion(*x*, *y*, *delta_x*, *delta_y*)

Met à jour la position globale de la souris.

Args:

x: Position horizontale actuelle de la souris. *y*: Position verticale actuelle de la souris. *delta_x*: Variation de la position horizontale. *delta_y*: Variation de la position verticale.

Parameters

- **x** (*int*)
- **y** (*int*)
- **delta_x** (*int*)
- **delta_y** (*int*)

Return type

None

on_mouse_press(*x, y, button, key_modifiers*)

Gère la sélection de l'interface selon les coordonnées du clic.

Args:

x: Coordonnée horizontale du clic. y: Coordonnée verticale du clic. button: Identifiant du bouton de la souris. key_modifiers: Modificateurs de touches actifs (opérations bit à bit).

Parameters

- **x** (*int*)
- **y** (*int*)
- **button** (*int*)
- **key_modifiers** (*int*)

Return type

None

on_mouse_release(*x, y, button, key_modifiers*)

Gère les événements de relâchement du bouton de la souris.

Args:

x: Coordonnée horizontale du relâchement. y: Coordonnée verticale du relâchement. button: Identifiant du bouton de la souris. key_modifiers: Modificateurs de touches actifs (opérations bit à bit).

Parameters

- **x** (*int*)
- **y** (*int*)
- **button** (*int*)
- **key_modifiers** (*int*)

Return type

None

on_update(*delta_time*)

Met à jour la logique à chaque image (frame).

Args:

delta_time: Temps écoulé depuis la dernière image.

Parameters

delta_time (*float*)

Return type

None

reset()

Réinitialise l'état de la vue.

Return type

None

setup()

Configure les éléments initiaux de l'interface et liste les puces disponibles.

Return type

None

class `modules.ui.editor.view.EditorView`(*id=None, level=None, level_solution=None*)

Orchestre le flux de travail de l'édition de circuits, y compris les composants UI, la manipulation des portes et les cycles de rendu.

Parameters

- **id** (*str* / *None*)
- **level** (*Any* / *None*)
- **level_solution** (*any* / *None*)

bottom_bar_update_camera()

Synchronise la position de défilement des portes de la barre inférieure.

Return type

None

bottom_bar_width_sum()

Calcule l'espace horizontal total requis pour toutes les portes de la barre inférieure.

Return type

int

bottom_gate_bar()

Actualise le contenu de la barre UI inférieure en fonction de la catégorie active.

Return type

None

property camera: `Tuple[int, int]`

Récupère les coordonnées actuelles de la caméra.

delete()

Initie le processus de suppression pour l'objet actuellement sélectionné.

Return type

None

delete_gate(id)

Supprime une porte et retire tous les chemins associés à ses ports.

Args:

id: L'identifiant de la porte à supprimer.

Parameters

id (*str*)

Return type

None

delete_path(*path_id*)

Supprime un chemin spécifique par son identifiant.

draw_bottom_gates()

Rend le menu de sélection des portes.

Return type

None

draw_debug_text()

Rend les informations de diagnostic pour le suivi des performances.

Return type

None

draw_frame_background()

Rend l'arrière-plan de l'espace de travail basé sur une grille.

Return type

None

draw_frame_border()

Rend la bordure extérieure de la fenêtre UI et son arrière-plan.

Return type

None

draw_frame_border_no_bg()

Rend les éléments de premier plan de la bordure de la fenêtre UI.

Return type

None

draw_tile(*id*, *x*, *y*)

Rend un segment de bordure UI individuel.

Args:

id: L'identifiant de la texture de tuile spécifique. *x*: Position horizontale. *y*: Position verticale.

Parameters

- *id* (*Any*)
- *x* (*float*)
- *y* (*float*)

Return type

None

get_hovered_bottom_gate()

Identifie quelle porte de la barre inférieure est actuellement survolée, le cas échéant.

Returns:

Un tuple contenant le flag de catégorie et l'identifiant correspondant.

Return type

Tuple[*int*, *Any*]

on_draw()

Exécute la séquence principale de rendu de la trame.

Return type

None

on_key_press(*key*, *key_modifiers*)

Gère les actions déclenchées par le clavier pour le basculement d'entrée, la navigation et l'édition.

Parameters

- **key** (*int*)
- **key_modifiers** (*int*)

Return type

None

on_key_release(*key*, *key_modifiers*)

Gère les événements de relâchement de touche.

Parameters

- **key** (*int*)
- **key_modifiers** (*int*)

Return type

None

on_mouse_motion(*x*, *y*, *delta_x*, *delta_y*)

Met à jour les traceurs de souris UI et effectue les calculs de déplacement.

Parameters

- **x** (*float*)
- **y** (*float*)
- **delta_x** (*float*)
- **delta_y** (*float*)

Return type

None

on_mouse_press(*x*, *y*, *button*, *key_modifiers*)

Gère les événements de clic pour la sélection UI et les interactions avec l'espace de travail.

Parameters

- **x** (*float*)
- **y** (*float*)
- **button** (*int*)
- **key_modifiers** (*int*)

Return type

None

on_mouse_release(*x*, *y*, *button*, *key_modifiers*)

Finalise les opérations de glisser-déposer pour les portes et le mouvement de caméra.

Parameters

- **x** (*float*)
- **y** (*float*)
- **button** (*int*)
- **key_modifiers** (*int*)

Return type

None

on_mouse_scroll(*x*, *y*, *scroll_x*, *scroll_y*)

Gère le défilement à l'intérieur de la barre UI inférieure.

Parameters

- **x** (*float*)
- **y** (*float*)
- **scroll_x** (*float*)
- **scroll_y** (*float*)

Return type

None

on_update(*delta_time*)

Exécute la logique de simulation et met à jour les métriques de performance.

Parameters**delta_time** (*float*)**Return type**

None

reset()

Réinitialise l'état de la vue.

Return type

None

setup_editor_categories()

Configure les entités UI représentant les catégories de sélection de portes.

Return type

None

simulate()

Invoque le moteur logique pour propager les valeurs des signaux.

Return type

None

Level Editor .. automodule:: modules.ui.level_editor**members**

Fournit la vue SaveFrame pour l'édition et la sauvegarde des données de configuration de niveau.

class modules.ui.level_editor.save.**SaveFrame**(*level*)

Gère la disposition de l'interface utilisateur et les interactions pour modifier les propriétés d'un niveau.

Parameters**level** (*Any*)

get_save_gate_counts()

Calcule la fréquence des types de portes utilisés dans le niveau actuel.

Returns:

Un dictionnaire associant les types de portes spécifiques à leur nombre total d'occurrences.

Return type

Dict[Any, int]

on_draw()

Affiche tous les éléments textuels configurés de l'interface utilisateur.

Return type

None

on_key_press(key, key_modifiers)

Gère les événements de pression de touches du clavier.

Args:

key: L'identifiant de la touche pressée. key_modifiers: Drapeaux binaires pour les touches de modification.

Parameters

- **key** (*int*)
- **key_modifiers** (*int*)

Return type

None

on_key_release(key, key_modifiers)

Gère les événements de relâchement de touches.

Parameters

- **key** (*int*)
- **key_modifiers** (*int*)

Return type

None

on_mouse_motion(x, y, delta_x, delta_y)

Met à jour l'état global du suivi de la souris.

Args:

x: La coordonnée x actuelle de la souris. y: La coordonnée y actuelle de la souris. delta_x: La variation de la coordonnée x depuis la dernière image. delta_y: La variation de la coordonnée y depuis la dernière image.

Parameters

- **x** (*float*)
- **y** (*float*)
- **delta_x** (*float*)
- **delta_y** (*float*)

Return type

None

on_mouse_press(*x*, *y*, *button*, *key_modifiers*)

Traite les interactions de clic de souris avec les éléments de l'interface.

Args:

x: La coordonnée *x* du clic de souris. *y*: La coordonnée *y* du clic de souris. *button*: Le bouton de la souris pressé. *key_modifiers*: Drapeaux binaires pour les touches de modification.

Parameters

- ***x*** (*float*)
- ***y*** (*float*)
- ***button*** (*int*)
- ***key_modifiers*** (*int*)

Return type

None

on_mouse_release(*x*, *y*, *button*, *key_modifiers*)

Gère les événements de relâchement du bouton de la souris.

Parameters

- ***x*** (*float*)
- ***y*** (*float*)
- ***button*** (*int*)
- ***key_modifiers*** (*int*)

Return type

None

on_update(*delta_time*)

Gère les mises à jour logiques périodiques.

Parameters***delta_time*** (*float*)**Return type**

None

reset()

Réinitialise l'état interne de la vue.

Return type

None

setup()

Initialise et positionne les éléments textuels de l'interface basés sur les données actuelles du niveau.

Return type

None

class `modules.ui.level_editor.selector.LevelEditorSelector`

Gère la vue de l'interface utilisateur pour parcourir les niveaux existants ou en créer de nouveaux.

on_draw()

Rendu de tous les éléments de texte de l'interface utilisateur et de leurs zones de collision (hitboxes) associées.

Return type

None

on_key_press(*key*, *key_modifiers*)

Gère les événements d'entrée au clavier.

Args:

key: Identifiant entier de la touche pressée. *key_modifiers*: Masque de bits des touches de modification actuellement maintenues.

Parameters

- **key** (*int*)
- **key_modifiers** (*int*)

Return type

None

on_key_release(*key*, *key_modifiers*)

Gère les événements de relâchement de touches.

Args:

key: Identifiant entier de la touche relâchée. *key_modifiers*: Masque de bits des touches de modification actuellement maintenues.

Parameters

- **key** (*int*)
- **key_modifiers** (*int*)

Return type

None

on_mouse_motion(*x*, *y*, *delta_x*, *delta_y*)

Met à jour l'état global du suivi de la souris.

Args:

x: Position horizontale actuelle de la souris. *y*: Position verticale actuelle de la souris. *delta_x*: Changement de position en x. *delta_y*: Changement de position en y.

Parameters

- **x** (*float*)
- **y** (*float*)
- **delta_x** (*float*)
- **delta_y** (*float*)

Return type

None

on_mouse_press(*x*, *y*, *button*, *key_modifiers*)

Déclenche la navigation de l'interface en fonction de l'interaction avec les zones de collision des éléments de texte.

Args:

x: Position horizontale actuelle de la souris. *y*: Position verticale actuelle de la souris. *button*: Identifiant du bouton de la souris pressé. *key_modifiers*: Masque de bits des touches de modification actuellement maintenues.

Parameters

- **x** (*float*)
- **y** (*float*)
- **button** (*int*)
- **key_modifiers** (*int*)

Return type

None

on_mouse_release(*x*, *y*, *button*, *key_modifiers*)

Gère les événements de relâchement du bouton de la souris.

Args:

x: Position horizontale actuelle de la souris. *y*: Position verticale actuelle de la souris. *button*: Identifiant du bouton de la souris relâché. *key_modifiers*: Masque de bits des touches de modification actuellement maintenues.

Parameters

- **x** (*float*)
- **y** (*float*)
- **button** (*int*)
- **key_modifiers** (*int*)

Return type

None

on_mouse_scroll(*x*, *y*, *scroll_x*, *scroll_y*)

Met à jour le décalage vertical de la caméra et reconstruit la mise en page.

Parameters

- **x** (*float*)
- **y** (*float*)
- **scroll_x** (*float*)
- **scroll_y** (*float*)

Return type

None

on_update(*delta_time*)

Met à jour la logique d'état à chaque image.

Args:

delta_time: Temps écoulé depuis la mise à jour précédente.

Parameters

delta_time (*float*)

Return type

None

reset()

Réinitialise l'état actuel de la vue.

Return type

None

setup()

Remplit les étiquettes de l'interface utilisateur et initialise les objets d'interaction pour les niveaux.

Return type

None

Ce module définit la classe `LevelEditorView`, qui fournit l'interface graphique principale pour la construction et la gestion des niveaux de circuits dans l'application.

class `modules.ui.level_editor.view.LevelEditorView(id=None)`

Gère l'interface de l'éditeur de niveaux, incluant le placement des portes, la création de chemins, le mouvement de la caméra et la logique d'interaction.

Parameters

id (*str* / *None*)

bottom_bar_width_sum()

Calcule la largeur cumulative de toutes les portes présentes dans la barre d'outils.

Retourne:

La largeur totale en pixels.

Return type

int

bottom_gate_bar()

Remplit la barre d'outils inférieure avec les types de portes disponibles.

Return type

None

property camera: `Tuple[int, int]`

Retourne le décalage actuel de la caméra.

delete()

Initie la suppression de l'objet actuellement sous le curseur.

Return type

None

delete_gate(*id*)

Supprime une porte et élague tous les chemins orphelins.

Args:

id: L'identifiant unique de la porte à supprimer.

Parameters

id (*str*)

Return type

None

draw_bottom_gates()

Affiche les éléments de l'interface utilisateur dans la barre d'outils inférieure.

Return type

None

draw_debug_text()

Affiche les métriques de performance et les informations du niveau actuel.

Return type

None

draw_frame_background()

Affiche l'arrière-plan de la grille de l'éditeur.

Return type

None

draw_frame_border()

Affiche la superposition de l'interface de bordure de la fenêtre de l'éditeur.

Return type

None

draw_tile(*id*, *x*, *y*)

Affiche une tuile de bordure individuelle.

Args:

id: L'identifiant unique de la texture. *x*: Coordonnée horizontale. *y*: Coordonnée verticale.

Parameters

- *id* (*str*)
- *x* (*int*)
- *y* (*int*)

Return type

None

get_hovered_bottom_gate()

Identifie si le curseur survole une porte spécifique dans la barre d'outils.

Retourne:

Le type de la porte survolée, ou None si aucune porte n'est sous le curseur.

Return type
str | None

on_draw()

Affiche le circuit actif et la superposition de l'interface utilisateur.

Return type
None

on_key_press(key, key_modifiers)

Traite les entrées clavier pour les commandes de l'éditeur.

Args:

key: Le code numérique de la touche pressée. key_modifiers: Drapeaux binaires pour les touches de modification.

Parameters

- **key** (*int*)
- **key_modifiers** (*int*)

Return type
None

on_key_release(key, key_modifiers)

Gère les événements de relâchement de touches.

Parameters

- **key** (*int*)
- **key_modifiers** (*int*)

Return type
None

on_mouse_motion(x, y, delta_x, delta_y)

Gère les mises à jour des interactions de la souris, incluant le glissement et le mouvement.

Args:

x: Position horizontale de la souris. y: Position verticale de la souris. delta_x: Changement horizontal. delta_y: Changement vertical.

Parameters

- **x** (*int*)
- **y** (*int*)
- **delta_x** (*int*)
- **delta_y** (*int*)

Return type
None

on_mouse_press(x, y, button, key_modifiers)

Gère les déclenchements d'interaction de la souris pour la construction de chemins et le placement de composants.

Parameters

- **x** (*int*)
- **y** (*int*)
- **button** (*int*)
- **key_modifiers** (*int*)

Return type

None

on_mouse_release(*x, y, button, key_modifiers*)

Finalise les opérations de glissement de caméra ou de placement de porte.

Parameters

- **x** (*int*)
- **y** (*int*)
- **button** (*int*)
- **key_modifiers** (*int*)

Return type

None

on_update(*delta_time*)

Met à jour l'état de la simulation et les métriques temporelles.

Args:*delta_time*: Temps écoulé depuis la dernière mise à jour.**Parameters****delta_time** (*float*)**Return type**

None

reset()

Réinitialise ou efface l'état de l'éditeur.

Return type

None

save_frame()

Ouvre la fenêtre de dialogue de sauvegarde.

Return type

None

simulate()

Exécute la propagation de la logique du circuit.

Return type

None

Level List .. automodule:: modules.ui.level_list**members**

Fournit une vue pour la navigation et la sélection des niveaux de jeu.

class `modules.ui.level_list.view.LevelList`

Affiche une liste défilante et catégorisée de niveaux sélectionnables.

on_draw()

Affiche l'arrière-plan, les boutons de niveau, les étiquettes de catégorie et l'interface utilisateur.

Return type

None

on_key_press(*key*, *key_modifiers*)

Gère les entrées de navigation clavier.

Parameters

- **key** (*int*)
- **key_modifiers** (*int*)

Return type

None

on_key_release(*key*, *key_modifiers*)

Gère les événements de relâchement de touches.

Parameters

- **key** (*int*)
- **key_modifiers** (*int*)

Return type

None

on_mouse_motion(*x*, *y*, *delta_x*, *delta_y*)

Met à jour l'état global du suivi de la souris.

Parameters

- **x** (*float*)
- **y** (*float*)
- **delta_x** (*float*)
- **delta_y** (*float*)

Return type

None

on_mouse_press(*x*, *y*, *button*, *key_modifiers*)

Détecte les clics de sélection de niveau et lance la transition de scène.

Parameters

- **x** (*float*)
- **y** (*float*)
- **button** (*int*)
- **key_modifiers** (*int*)

Return type

None

on_mouse_release(*x*, *y*, *button*, *key_modifiers*)

Gère les événements de relâchement de la souris.

Parameters

- **x** (*float*)
- **y** (*float*)
- **button** (*int*)
- **key_modifiers** (*int*)

Return type

None

on_mouse_scroll(*x*, *y*, *scroll_x*, *scroll_y*)

Met à jour le décalage vertical de la caméra et reconstruit la disposition.

Parameters

- **x** (*float*)
- **y** (*float*)
- **scroll_x** (*float*)
- **scroll_y** (*float*)

Return type

None

on_update(*delta_time*)

Effectue les mises à jour logiques à chaque image.

Parameters

- delta_time** (*float*)

Return type

None

reset()

Réinitialise la vue à son état initial.

Return type

None

setup()

Construit la disposition de l'interface, organise les niveaux par catégories et initialise les boutons.

Return type

None

Level Player .. automodule:: modules.ui.level_player

members

class modules.ui.level_player.selector.**LevelPlayerSelector**

Fournit une interface permettant aux utilisateurs de sélectionner et de lancer les niveaux du jeu.

on_draw()

Rendu des éléments textuels et de leurs zones de collision (hitboxes) associées.

Return type

None

on_key_press(*key*, *key_modifiers*)**Args:**

key: Identifiant entier de la touche pressée. *key_modifiers*: Masque de bits des touches de modification actives.

Parameters

- **key** (*int*)
- **key_modifiers** (*int*)

Return type

None

on_key_release(*key*, *key_modifiers*)**Args:**

key: Identifiant entier de la touche relâchée. *key_modifiers*: Masque de bits des touches de modification actives.

Parameters

- **key** (*int*)
- **key_modifiers** (*int*)

Return type

None

on_mouse_motion(*x*, *y*, *delta_x*, *delta_y*)**Args:**

x: Position horizontale actuelle de la souris. *y*: Position verticale actuelle de la souris. *delta_x*: Variation de la position horizontale. *delta_y*: Variation de la position verticale.

Parameters

- **x** (*float*)
- **y** (*float*)
- **delta_x** (*float*)
- **delta_y** (*float*)

Return type

None

on_mouse_press(*x*, *y*, *button*, *key_modifiers*)**Args:**

x: Position horizontale de la souris lors du clic. *y*: Position verticale de la souris lors du clic. *button*: Identifiant du bouton de la souris pressé. *key_modifiers*: Masque de bits des touches de modification actives.

Parameters

- **x** (*float*)
- **y** (*float*)
- **button** (*int*)
- **key_modifiers** (*int*)

Return type

None

on_mouse_release(*x, y, button, key_modifiers*)**Args:**

x: Position horizontale de la souris. y: Position verticale de la souris. button: Identifiant du bouton de la souris relâché. key_modifiers: Masque de bits des touches de modification actives.

Parameters

- **x** (*float*)
- **y** (*float*)
- **button** (*int*)
- **key_modifiers** (*int*)

Return type

None

on_mouse_scroll(*x, y, scroll_x, scroll_y*)

Met à jour le décalage vertical de la caméra et reconstruit la disposition.

Parameters

- **x** (*float*)
- **y** (*float*)
- **scroll_x** (*float*)
- **scroll_y** (*float*)

Return type

None

on_update(*delta_time*)**Args:**

delta_time: Temps écoulé depuis la dernière mise à jour de l'image.

Parameters

delta_time (*float*)

Return type

None

reset()

Réinitialise l'état de la vue.

Return type

None

setup()

Configure la disposition de l'interface et associe les niveaux disponibles aux éléments sélectionnables.

Return type

None

class `modules.ui.level_player.view.LevelPlayer`(*id=None*)

Manages the interactive game level view, including UI rendering, user-driven editing, and logic gate simulation.

Parameters

id (*str* / *None*)

bottom_bar_update_camera()

Syncs the scroll position of bottom bar gates.

Return type

None

bottom_bar_width_sum()

Returns the aggregate width of all gate icons in the bottom toolbar.

Return type

int

bottom_gate_bar()

Populates the bottom UI toolbar with available gates based on level constraints.

Return type

None

property camera: `Tuple[int, int]`

Provides the current quantized camera position.

delete()

Initiates the deletion process for the currently selected object.

Return type

None

delete_gate(id)

Removes specified gate and reconciles affected circuit paths.

Parameters

id (*str*)

Return type

None

draw_bottom_gates()

Renders the gate icons in the bottom toolbar.

Return type

None

draw_debug_text()

Renders engine performance metrics and object counts.

Return type

None

draw_frame_background()

Renders the level grid background texture.

Return type

None

draw_frame_border()

Renders the outer UI window boundary.

Return type

None

draw_frame_border_top()

Renders the outer UI window boundary top (second layer to prevent clipping).

Return type

None

draw_hint()

Render the hints frame, to provide basic level instructions.

Return type

None

draw_level_time()

Renders current elapsed time versus limits and progress markers.

Return type

None

draw_right()

Renders the sidebar components and calculated truth table.

Return type

None

draw_won()

Renders the victory modal and animated star icons.

Return type

None

get_hovered_bottom_gate()

Identifies which gate icon in the toolbar is currently being hovered.

Return type

str | None

launch_next_level()

Transitions application to the subsequent level in the sequence.

Return type

None

on_draw()

Main rendering loop: updates component visual states and draws layers.

Return type

None

on_key_press(*key*, *key_modifiers*)

Handles keyboard shortcuts for interaction, deletion, and navigation.

Parameters

- **key** (*int*)
- **key_modifiers** (*int*)

Return type

None

on_key_release(*key*, *key_modifiers*)

Handles key-up events.

Parameters

- **key** (*int*)
- **key_modifiers** (*int*)

Return type

None

on_mouse_motion(*x*, *y*, *delta_x*, *delta_y*)

Handles mouse movement for dragging objects and updating circuit layout.

Parameters

- **x** (*int*)
- **y** (*int*)
- **delta_x** (*int*)
- **delta_y** (*int*)

Return type

None

on_mouse_press(*x*, *y*, *button*, *key_modifiers*)

Handles click events for UI navigation, wiring paths, and gate placement.

Parameters

- **x** (*int*)
- **y** (*int*)
- **button** (*int*)
- **key_modifiers** (*int*)

Return type

None

on_mouse_release(*x*, *y*, *button*, *key_modifiers*)

Handles mouse button release interactions.

Parameters

- **x** (*int*)
- **y** (*int*)
- **button** (*int*)

- **key_modifiers** (*int*)

Return type

None

on_mouse_scroll(*x*, *y*, *scroll_x*, *scroll_y*)

Handles scrolling within the bottom bar UI.

Parameters

- **x** (*float*)
- **y** (*float*)
- **scroll_x** (*float*)
- **scroll_y** (*float*)

Return type

None

on_update(*delta_time*)

Per-frame update loop for simulation and state tracking.

Parameters**delta_time** (*float*)**Return type**

None

prepare_hint_frame()

Initializes UI elements for the hints screen. (Level Instructions.)

Return type

None

prepare_right_frame_basic()

Constructs the right-hand sidebar UI, including the truth table rendering.

Return type

None

prepare_won_frame()

Initializes UI elements for the victory screen.

Return type

None

reset()

Resets the current level state.

Return type

None

simulate()

Propagates logic states through the current circuit chip.

Return type

None

won()

Activates victory-related UI and logic states.

Return type

None

Loading .. automodule:: modules.ui.loading.view**members****Main Menu** .. automodule:: modules.ui.main_menu**members****class** modules.ui.main_menu.in_progress_view.**MainMenuView**(*pause=False*)

Gère l'interface du menu principal, incluant le rendu de l'UI et la navigation.

add_paths()

Initialise et enregistre les structures de chemins pour les visuels d'arrière-plan du menu.

Return type

None

draw_frame_background()

Remplit la zone d'arrière-plan interne du menu.

Return type

None

draw_frame_border()

Construit la bordure de l'UI en utilisant un rendu de tuiles répétitif.

Return type

None

draw_paths()

Affiche tous les visuels de chemins initialisés.

Return type

None

draw_tile(*id, x, y*)

Affiche une seule tuile de bordure d'UI depuis le registre de données.

Args:

id: L'index de la texture de la tuile. *x*: La coordonnée horizontale de l'écran. *y*: La coordonnée verticale de l'écran.

Parameters

- *id* (*int*)
- *x* (*int*)
- *y* (*int*)

Return type

None

on_draw()

Phase de rendu de la scène du menu.

Return type

None

on_key_press(*key*, *key_modifiers*)

Traite les entrées clavier pour les commandes du menu.

Args:

key: Le code numérique de la touche. *key_modifiers*: Masque de bits des touches de modification.

Parameters

- **key** (*int*)
- **key_modifiers** (*int*)

Return type

None

on_mouse_motion(*x*, *y*, *delta_x*, *delta_y*)

Gère le suivi de la souris, les états de survol de l'UI et la progression de l'easter egg.

Args:

x: Coordonnée X actuelle du curseur. *y*: Coordonnée Y actuelle du curseur. *delta_x*: Déplacement horizontal de la souris. *delta_y*: Déplacement vertical de la souris.

Parameters

- **x** (*float*)
- **y** (*float*)
- **delta_x** (*float*)
- **delta_y** (*float*)

Return type

None

on_mouse_press(*x*, *y*, *button*, *key_modifiers*)

Gère les événements d'interaction pour la navigation dans le menu.

Args:

x: Coordonnée X du clic. *y*: Coordonnée Y du clic. *button*: Le bouton de la souris pressé. *key_modifiers*: Modificateurs de clavier actifs.

Parameters

- **x** (*float*)
- **y** (*float*)
- **button** (*int*)
- **key_modifiers** (*int*)

Return type

None

rainbow_color(*precision*, *index*)

Calcule une couleur hexadécimale basée sur un cycle de valeurs HSV.

Args:

precision: La fréquence de la transition de couleur. *index*: L'étape actuelle de l'animation.

Returns:

La chaîne hexadécimale résultante.

Parameters

- **precision** (*int*)
- **index** (*int*)

Return type

str

class modules.ui.main_menu.settings_view.**SettingView**

Gère l'interface du menu des paramètres, incluant la disposition de l'interface utilisateur et les interactions avec l'utilisateur.

on_draw()

Effectue le rendu de la pile complète des vues.

Return type

None

on_key_press(*key, key_modifiers*)

Traite les entrées clavier.

Args:

key: Le code numérique de la touche pressée. key_modifiers: Drapeaux binaires pour les touches de modification maintenues.

Parameters

- **key** (*int*)
- **key_modifiers** (*int*)

Return type

None

on_mouse_motion(*x, y, delta_x, delta_y*)

Synchronise l'état global de la souris avec la position actuelle du curseur.

Args:

x: Position horizontale actuelle du curseur. y: Position verticale actuelle du curseur. delta_x: Mouvement horizontal relatif à la dernière image. delta_y: Mouvement vertical relatif à la dernière image.

Parameters

- **x** (*float*)
- **y** (*float*)
- **delta_x** (*float*)
- **delta_y** (*float*)

Return type

None

on_mouse_press(*x*, *y*, *button*, *key_modifiers*)

Traite les clics de souris pour déclencher la navigation ou des événements d'action.

Args:

x: Position horizontale du curseur au moment du clic. *y*: Position verticale du curseur au moment du clic. *button*: Le bouton spécifique actionné. *key_modifiers*: Drapeaux binaires pour les touches de modification maintenues.

Parameters

- **x** (*float*)
- **y** (*float*)
- **button** (*int*)
- **key_modifiers** (*int*)

Return type

None

on_mouse_scroll(*x*, *y*, *scroll_x*, *scroll_y*)

Met à jour le décalage vertical de la caméra et reconstruit la mise en page.

Parameters

- **x** (*float*)
- **y** (*float*)
- **scroll_x** (*float*)
- **scroll_y** (*float*)

Return type

None

class `modules.ui.main_menu.tutorial_view.TutorialView`

Gère la disposition, le rendu et la logique d'interaction pour l'écran du tutoriel.

on_draw()

Affiche l'état actuel de la vue à l'écran.

Return type

None

on_key_press(*key*, *key_modifiers*)

Gère les événements de saisie au clavier.

Args:

key: Le code de la touche pressée. *key_modifiers*: Drapeaux binaires indiquant les touches de modification actives.

Parameters

- **key** (*int*)
- **key_modifiers** (*int*)

Return type

None

on_mouse_motion(*x*, *y*, *delta_x*, *delta_y*)

Met à jour l'état global du suivi de la souris.

Args:

x: Position horizontale actuelle de la souris. *y*: Position verticale actuelle de la souris. *delta_x*: Mouvement horizontal depuis la dernière image. *delta_y*: Mouvement vertical depuis la dernière image.

Parameters

- ***x*** (*float*)
- ***y*** (*float*)
- ***delta_x*** (*float*)
- ***delta_y*** (*float*)

Return type

None

on_mouse_press(*x*, *y*, *button*, *key_modifiers*)

Traite les événements d'interaction de la souris pour les éléments de l'interface.

Args:

x: Position horizontale de la souris au moment du clic. *y*: Position verticale de la souris au moment du clic. *button*: Le bouton de la souris pressé. *key_modifiers*: Drapeaux binaires pour les touches de modification actives.

Parameters

- ***x*** (*float*)
- ***y*** (*float*)
- ***button*** (*int*)
- ***key_modifiers*** (*int*)

Return type

None

on_mouse_scroll(*x*, *y*, *scroll_x*, *scroll_y*)

Met à jour le décalage vertical de la caméra et reconstruit la mise en page.

Parameters

- ***x*** (*float*)
- ***y*** (*float*)
- ***scroll_x*** (*float*)
- ***scroll_y*** (*float*)

Return type

None

class `modules.ui.main_menu.view_menu.GameView`

Gère la disposition, le rendu et la logique d'interaction du menu principal.

on_draw()

Affiche les composants de l'interface utilisateur à l'écran.

Return type

None

on_key_press(*key*, *key_modifiers*)**Arguments:**

key: L'identifiant numérique de la touche pressée. *key_modifiers*: Drapeaux binaires représentant les touches de modification actives.

Parameters

- **key** (*int*)
- **key_modifiers** (*int*)

Return type

None

on_key_release(*key*, *key_modifiers*)**Arguments:**

key: L'identifiant numérique de la touche relâchée. *key_modifiers*: Drapeaux binaires représentant les touches de modification actives.

Parameters

- **key** (*int*)
- **key_modifiers** (*int*)

Return type

None

on_mouse_motion(*x*, *y*, *delta_x*, *delta_y*)**Arguments:**

x: Coordonnée horizontale de la souris à l'écran. *y*: Coordonnée verticale de la souris à l'écran. *delta_x*: Variation de la position horizontale. *delta_y*: Variation de la position verticale.

Parameters

- **x** (*float*)
- **y** (*float*)
- **delta_x** (*float*)
- **delta_y** (*float*)

Return type

None

on_mouse_press(*x*, *y*, *button*, *key_modifiers*)**Arguments:**

x: Coordonnée horizontale du clic à l'écran. *y*: Coordonnée verticale du clic à l'écran. *button*: Le bouton de la souris pressé. *key_modifiers*: Drapeaux binaires représentant les touches de modification actives.

Parameters

- **x** (*float*)
- **y** (*float*)
- **button** (*int*)
- **key_modifiers** (*int*)

Return type

None

on_mouse_release(*x, y, button, key_modifiers*)**Arguments:**

x: Coordonnée horizontale du relâchement à l'écran. **y**: Coordonnée verticale du relâchement à l'écran. **button**: Le bouton de la souris relâché. **key_modifiers**: Drapeaux binaires représentant les touches de modification actives.

Parameters

- **x** (*float*)
- **y** (*float*)
- **button** (*int*)
- **key_modifiers** (*int*)

Return type

None

on_update(*delta_time*)

Met à jour la logique de la vue.

Parameters**delta_time** (*float*)**Return type**

None

reset()

Réinitialise l'état de la vue.

Return type

None

Saves List .. automodule:: modules.ui.saves_list.view**members**

8.4.2 Toolbox

Fournit une implémentation de bouton UI avec gestion géométrique et zones de collision (hitboxes) d'interaction.

class modules.ui.toolbox.button.**Button**

Représente un élément de bouton UI avec positionnement, dimensions et fonctionnalités de zone de collision.

property anchor: Vec2

Retourne le vecteur d'ancrage actuel.

property color: <module 'arcade.color' from 'C:\\Users\\soubrie\\AppData\\Roaming\\Python\\Python313\\site-packages\\arcade\\color__init__.py'>

Retourne la couleur principale du bouton.

draw()

Rendu du texte et de la hitbox en fonction de l'échelle actuelle et des contraintes de la grille.

Return type

None

property height: float

Retourne la hauteur du bouton.

property name: str

Retourne le nom de l'étiquette du bouton.

property text: Text

Retourne l'objet texte arcade sous-jacent.

property touched: bool

Retourne l'état d'interaction provenant de la hitbox.

property width: float

Retourne la largeur du bouton.

property x: float

Retourne la position horizontale.

property y: float

Retourne la position verticale.

Une bibliothèque fournissant diverses fonctions d'atténuation (easing) mathématiques pour les séquences d'animation.

Ce module implémente des courbes d'interpolation standard pour créer des transitions fluides et naturelles entre une valeur de départ et une valeur de fin sur une durée définie.

class modules.ui.toolbox.easing.BackEaseIn(start=0, end=1, duration=100)

Effectue une atténuation de type back-in, avec un léger dépassement au début.

Parameters

- **start** (*float*)
- **end** (*float*)
- **duration** (*int*)

func()

Calcule l'atténuation Back In.

Return type

float

class modules.ui.toolbox.easing.BackEaseInOut(start=0, end=1, duration=100)

Effectue une atténuation de type back-in-out avec des dépassements aux deux extrémités.

Parameters

- **start** (*float*)

- **end** (*float*)
- **duration** (*int*)

func()

Calcule l'atténuation Back In-Out.

Return type

float

class `modules.ui.toolbox.easing.BackEaseOut` (*start=0, end=1, duration=100*)

Effectue une atténuation de type back-out, avec un léger dépassement à la fin.

Parameters

- **start** (*float*)
- **end** (*float*)
- **duration** (*int*)

func()

Calcule l'atténuation Back Out.

Return type

float

class `modules.ui.toolbox.easing.BounceEaseIn` (*start=0, end=1, duration=100*)

Effectue une atténuation de type rebond (bounce-in).

Parameters

- **start** (*float*)
- **end** (*float*)
- **duration** (*int*)

func()

Calcule l'atténuation Bounce In en déléguant à BounceEaseOut.

Return type

float

class `modules.ui.toolbox.easing.BounceEaseInOut` (*start=0, end=1, duration=100*)

Effectue une atténuation de type rebond (bounce-in-out).

Parameters

- **start** (*float*)
- **end** (*float*)
- **duration** (*int*)

func()

Calcule l'atténuation Bounce In-Out.

Return type

float

class `modules.ui.toolbox.easing.BounceEaseOut` (*start=0, end=1, duration=100*)

Effectue une atténuation de type rebond (bounce-out).

Parameters

- **start** (*float*)
- **end** (*float*)
- **duration** (*int*)

func()

Calcule l'atténuation Bounce Out en utilisant des fonctions par morceaux.

Return type
float

class modules.ui.toolbox.easing.**CircularEaseIn**(*start=0, end=1, duration=100*)

Effectue une interpolation circulaire de type ease-in.

Parameters

- **start** (*float*)
- **end** (*float*)
- **duration** (*int*)

func()

Calcule l'atténuation circulaire In.

Return type
float

class modules.ui.toolbox.easing.**CircularEaseInOut**(*start=0, end=1, duration=100*)

Effectue une interpolation circulaire de type ease-in-out.

Parameters

- **start** (*float*)
- **end** (*float*)
- **duration** (*int*)

func()

Calcule l'atténuation circulaire In-Out.

Return type
float

class modules.ui.toolbox.easing.**CircularEaseOut**(*start=0, end=1, duration=100*)

Effectue une interpolation circulaire de type ease-out.

Parameters

- **start** (*float*)
- **end** (*float*)
- **duration** (*int*)

func()

Calcule l'atténuation circulaire Out.

Return type
float

class modules.ui.toolbox.easing.**CubicEaseIn**(*start=0, end=1, duration=100*)

Effectue une interpolation cubique de type ease-in.

Parameters

- **start** (*float*)
- **end** (*float*)
- **duration** (*int*)

func()

Calcule l'atténuation cubique In.

Return type

float

class modules.ui.toolbox.easing.**CubicEaseInOut**(*start=0, end=1, duration=100*)

Effectue une interpolation cubique de type ease-in-out.

Parameters

- **start** (*float*)
- **end** (*float*)
- **duration** (*int*)

func()

Calcule l'atténuation cubique In-Out.

Return type

float

class modules.ui.toolbox.easing.**CubicEaseOut**(*start=0, end=1, duration=100*)

Effectue une interpolation cubique de type ease-out.

Parameters

- **start** (*float*)
- **end** (*float*)
- **duration** (*int*)

func()

Calcule l'atténuation cubique Out.

Return type

float

class modules.ui.toolbox.easing.**EasingBase**(*start=0, end=1, duration=100*)

Interface de base pour l'implémentation de transitions d'atténuation spécifiques.

Parameters

- **start** (*float*)
- **end** (*float*)
- **duration** (*int*)

func()

Calcule le facteur de progression normalisé.

Returns:

Le scalaire de progression entre 0.0 et 1.0.

Return type

float

reset()

Réinitialise l'état de l'animation à la position initiale.

Return type

None

tick()

Avance l'état interne d'une étape et calcule la valeur actuelle.

Returns:

La valeur interpolée à l'étape actuelle de la chronologie.

Return type

float

class modules.ui.toolbox.easing.**ElasticEaseIn**(*start=0, end=1, duration=100*)

Effectue une interpolation élastique de type ease-in.

Parameters

- **start** (*float*)
- **end** (*float*)
- **duration** (*int*)

func()

Calcule l'atténuation élastique In.

Return type

float

class modules.ui.toolbox.easing.**ElasticEaseInOut**(*start=0, end=1, duration=100*)

Effectue une interpolation élastique de type ease-in-out.

Parameters

- **start** (*float*)
- **end** (*float*)
- **duration** (*int*)

func()

Calcule l'atténuation élastique In-Out.

Return type

float

class modules.ui.toolbox.easing.**ElasticEaseOut**(*start=0, end=1, duration=100*)

Effectue une interpolation élastique de type ease-out.

Parameters

- **start** (*float*)
- **end** (*float*)
- **duration** (*int*)

func()

Calcule l'atténuation élastique Out.

Return type

float

class modules.ui.toolbox.easing.**ExponentialEaseIn**(*start=0, end=1, duration=100*)

Effectue une interpolation exponentielle de type ease-in.

Parameters

- **start** (*float*)
- **end** (*float*)
- **duration** (*int*)

func()

Calcule l'atténuation exponentielle In.

Return type

float

class modules.ui.toolbox.easing.**ExponentialEaseInOut**(*start=0, end=1, duration=100*)

Effectue une interpolation exponentielle de type ease-in-out.

Parameters

- **start** (*float*)
- **end** (*float*)
- **duration** (*int*)

func()

Calcule l'atténuation exponentielle In-Out.

Return type

float

class modules.ui.toolbox.easing.**ExponentialEaseOut**(*start=0, end=1, duration=100*)

Effectue une interpolation exponentielle de type ease-out.

Parameters

- **start** (*float*)
- **end** (*float*)
- **duration** (*int*)

func()

Calcule l'atténuation exponentielle Out.

Return type

float

class modules.ui.toolbox.easing.**LinearInOut**(start=0, end=1, duration=100)

Effectue une interpolation linéaire entre le début et la fin.

Parameters

- **start** (float)
- **end** (float)
- **duration** (int)

func()

Calcule l'atténuation linéaire.

Return type

float

class modules.ui.toolbox.easing.**QuadEaseIn**(start=0, end=1, duration=100)

Effectue une interpolation quadratique de type ease-in.

Parameters

- **start** (float)
- **end** (float)
- **duration** (int)

func()

Calcule l'atténuation quadratique In.

Return type

float

class modules.ui.toolbox.easing.**QuadEaseInOut**(start=0, end=1, duration=100)

Effectue une interpolation quadratique de type ease-in-out.

Parameters

- **start** (float)
- **end** (float)
- **duration** (int)

func()

Calcule l'atténuation quadratique In-Out.

Return type

float

class modules.ui.toolbox.easing.**QuadEaseOut**(start=0, end=1, duration=100)

Effectue une interpolation quadratique de type ease-out.

Parameters

- **start** (float)
- **end** (float)

- **duration** (*int*)

func()

Calcule l'atténuation quadratique Out.

Return type

float

class modules.ui.toolbox.easing.**QuarticEaseIn**(*start=0, end=1, duration=100*)

Effectue une interpolation quartique de type ease-in.

Parameters

- **start** (*float*)
- **end** (*float*)
- **duration** (*int*)

func()

Calcule l'atténuation quartique In.

Return type

float

class modules.ui.toolbox.easing.**QuarticEaseInOut**(*start=0, end=1, duration=100*)

Effectue une interpolation quartique de type ease-in-out.

Parameters

- **start** (*float*)
- **end** (*float*)
- **duration** (*int*)

func()

Calcule l'atténuation quartique In-Out.

Return type

float

class modules.ui.toolbox.easing.**QuarticEaseOut**(*start=0, end=1, duration=100*)

Effectue une interpolation quartique de type ease-out.

Parameters

- **start** (*float*)
- **end** (*float*)
- **duration** (*int*)

func()

Calcule l'atténuation quartique Out.

Return type

float

class modules.ui.toolbox.easing.**QuinticEaseIn**(*start=0, end=1, duration=100*)

Effectue une interpolation quintique de type ease-in.

Parameters

- **start** (*float*)

- **end** (*float*)
- **duration** (*int*)

func()

Calcule l'atténuation quintique In.

Return type
float

class `modules.ui.toolbox.easing.QuinticEaseInOut`(*start=0, end=1, duration=100*)

Effectue une interpolation quintique de type ease-in-out.

Parameters

- **start** (*float*)
- **end** (*float*)
- **duration** (*int*)

func()

Calcule l'atténuation quintique In-Out.

Return type
float

class `modules.ui.toolbox.easing.QuinticEaseOut`(*start=0, end=1, duration=100*)

Effectue une interpolation quintique de type ease-out.

Parameters

- **start** (*float*)
- **end** (*float*)
- **duration** (*int*)

func()

Calcule l'atténuation quintique Out.

Return type
float

class `modules.ui.toolbox.easing.SineEaseIn`(*start=0, end=1, duration=100*)

Effectue une interpolation sinusoïdale de type ease-in.

Parameters

- **start** (*float*)
- **end** (*float*)
- **duration** (*int*)

func()

Calcule l'atténuation sinusoïdale In.

Return type
float

class `modules.ui.toolbox.easing.SineEaseInOut`(*start=0, end=1, duration=100*)

Effectue une interpolation sinusoïdale de type ease-in-out.

Parameters

- **start** (*float*)
- **end** (*float*)
- **duration** (*int*)

func()

Calcule l'atténuation sinusoïdale In-Out.

Return type

float

class `modules.ui.toolbox.easing.SineEaseOut`(*start=0, end=1, duration=100*)

Effectue une interpolation sinusoïdale de type ease-out.

Parameters

- **start** (*float*)
- **end** (*float*)
- **duration** (*int*)

func()

Calcule l'atténuation sinusoïdale Out.

Return type

float

class `modules.ui.toolbox.entity.Entity`(*x=0, y=0, width=10, height=10, sprite=None, anchor=(0, 0)*)

Gère les propriétés spatiales et l'état de rendu d'une entité.

Parameters

- **x** (*float*)
- **y** (*float*)
- **width** (*float*)
- **height** (*float*)
- **sprite** (*Sprite | None*)
- **anchor** (*Vec2*)

draw()

Rendu de l'entité en utilisant soit une forme primitive, soit un sprite.

Return type

None

property height: float

Retourne la hauteur de l'entité.

property touched: bool

Retourne l'état de collision actuel de la hitbox.

property width: float

Retourne la largeur de l'entité.

property x: float

Retourne la coordonnée X actuelle.

property y: float

Retourne la coordonnée Y actuelle.

Fournit des utilitaires de rendu de grille pour l'environnement de l'éditeur d'interface utilisateur.

class `modules.ui.toolbox.grid.Grid`

Gère la superposition de la grille de points visuelle pour l'interface de l'éditeur d'interface utilisateur.

draw()

Affiche une grille de points sur la zone de l'espace de travail de 1280x720.

Return type

None

Fournit une implémentation modulaire de hitbox pour la logique de collision et d'interaction.

class `modules.ui.toolbox.hitbox.HitBox`(*x=0, y=0, width=0, height=0, anchor=(0, 0)*)

Gère une zone rectangulaire pour la détection de collision et l'interaction UI.

Attributes:

`rect` (`arcade.XYWH`): La représentation interne du rectangle via arcade.

Parameters

- **x** (*float*)
- **y** (*float*)
- **width** (*float*)
- **height** (*float*)
- **anchor** (*Vec2*)

property anchor: Vec2

`arcade.Vec2`: Point d'ancrage actuel.

draw()

Affiche les limites de la hitbox avec un retour visuel pour l'état de survol.

Return type

None

property height: float

`float`: Hauteur de la zone.

property touched: bool

`bool`: Indique si la position actuelle de la souris intersecte la hitbox.

property width: float

`float`: Largeur de la zone.

property x: float

`float`: Coordonnée horizontale actuelle.

property y: float

`float`: Coordonnée verticale actuelle.

Module pour la génération d'identifiants uniques cryptographiquement sécurisés.

`modules.ui.toolbox.id_generator.random_id()`

Génère un identifiant hexadécimal de 128 bits.

Construit l'identifiant en hachant une combinaison d'un horodatage de 48 bits et d'une entropie aléatoire de 32 bits pour garantir l'unicité et l'imprévisibilité.

Returns:

Une chaîne hexadécimale de 32 caractères.

Return type

str

`modules.ui.toolbox.keys.apply_key(text, keycode, modifiers=0, int_only=False)`

Applique la saisie d'une touche à une chaîne de caractères existante en tenant compte des modificateurs (Majuscule, Verr. Maj) et des contraintes de type.

Parameters

- **text** (str)
- **keycode** (int)
- **modifiers** (int)

Return type

str

`modules.ui.toolbox.keys.visual_key(key_code)`

Récupère le nom symbolique d'une touche à partir de son code numérique en parcourant les attributs du module `arcade.key`.

class `modules.ui.toolbox.poly_hitbox.PolyHitbox(points=None)`

Gère les limites de collision basées sur des sommets et le rendu associé.

Parameters

points (List[Tuple[float, float]])

draw()

Affiche le contour du polygone à l'écran.

Return type

None

save()

Séréalise les données de la zone de collision pour la persistance.

Returns:

Un dictionnaire contenant l'identifiant et la collection de sommets.

Return type

Dict[str, Any]

property touched: bool

Détermine si le curseur de la souris chevauche la zone du polygone.

Returns:

True si les coordonnées de la souris intersectent la géométrie du polygone.

```
class modules.ui.toolbox.text.Text(x=0, y=0, width=10, height=10, text='Default Text', align=('center',
                                         'center'), size=18, multiline=False)
```

Gère le rendu du texte, le positionnement, l'alignement et la détection de collision.

Parameters

- **x** (*float*)
- **y** (*float*)
- **width** (*float*)
- **height** (*float*)
- **text** (*str*)
- **align** (*Tuple[str, str]*)
- **size** (*int*)
- **multiline** (*bool*)

property align: *Tuple[str, str]*

Retourne le tuple d'ancrage de l'alignement actuel.

property color: <module 'arcade.color' from 'C:\\Users\\soubrie\\AppData\\Roaming\\Python\\Python313\\site-packages\\arcade\\color__init__.py'>

Retourne la couleur actuelle du texte.

draw()

Effectue le rendu de l'élément texte.

Return type

None

property height: *float*

Retourne la hauteur actuelle de la zone.

property multiline: *bool*

Retourne si le support multiligne est activé.

property size: *int*

Retourne la taille actuelle de la police.

property text: *str*

Retourne le contenu actuel de la chaîne de caractères.

property touched: *bool*

Indique si l'élément fait actuellement l'objet d'une interaction.

property width: *float*

Retourne la largeur actuelle de la zone.

property x: *float*

Retourne la coordonnée X actuelle.

property y: *float*

Retourne la coordonnée Y actuelle.

PYTHON MODULE INDEX

m

- modules.data, 17
- modules.data.chip, 20
- modules.data.complex, 21
- modules.data.custom, 22
- modules.data.gate, 23
- modules.data.gate_index, 23
- modules.data.hint, 24
- modules.data.language, 25
- modules.data.level, 25
- modules.data.loader, 28
- modules.data.node, 31
- modules.data.nodes.eight.addr, 34
- modules.data.nodes.eight.gand, 34
- modules.data.nodes.eight.gnot, 34
- modules.data.nodes.eight.gor, 34
- modules.data.nodes.eight.input, 35
- modules.data.nodes.eight.nand, 35
- modules.data.nodes.eight.nor, 35
- modules.data.nodes.eight.one, 35
- modules.data.nodes.eight.output, 36
- modules.data.nodes.eight.sub, 36
- modules.data.nodes.eight.xor, 36
- modules.data.nodes.mix.eight_breaker, 36
- modules.data.nodes.mix.eight_maker, 36
- modules.data.nodes.one.addr, 36
- modules.data.nodes.one.clock, 37
- modules.data.nodes.one.delay, 37
- modules.data.nodes.one.gand, 37
- modules.data.nodes.one.gnot, 37
- modules.data.nodes.one.gor, 37
- modules.data.nodes.one.gpass, 37
- modules.data.nodes.one.input, 37
- modules.data.nodes.one.nand, 37
- modules.data.nodes.one.nor, 38
- modules.data.nodes.one.off, 38
- modules.data.nodes.one.on, 38
- modules.data.nodes.one.output, 38
- modules.data.nodes.one.sub, 38
- modules.data.nodes.one.swc, 38
- modules.data.nodes.one.tand, 38
- modules.data.nodes.one.tor, 39
- modules.data.nodes.one.xor, 39
- modules.data.nodes.path, 31
- modules.engine, 39
- modules.engine.logic, 39
- modules.logger, 48
- modules.ui, 49
- modules.ui.debug_display_all_tiles.view, 50
- modules.ui.editor.input, 52
- modules.ui.editor.save, 54
- modules.ui.editor.selector, 56
- modules.ui.editor.view, 58
- modules.ui.level_editor.save, 61
- modules.ui.level_editor.selector, 63
- modules.ui.level_editor.view, 66
- modules.ui.level_list.view, 69
- modules.ui.level_player.selector, 71
- modules.ui.level_player.view, 74
- modules.ui.main_menu.in_progress_view, 78
- modules.ui.main_menu.settings_view, 80
- modules.ui.main_menu.tutorial_view, 81
- modules.ui.main_menu.verification, 82
- modules.ui.main_menu.view_menu, 82
- modules.ui.mouse, 49
- modules.ui.toolbox.button, 84
- modules.ui.toolbox.easing, 85
- modules.ui.toolbox.entity, 94
- modules.ui.toolbox.grid, 95
- modules.ui.toolbox.hitbox, 95
- modules.ui.toolbox.id_generator, 95
- modules.ui.toolbox.keys, 96
- modules.ui.toolbox.poly_hitbox, 96
- modules.ui.toolbox.text, 96
- modules.ui.window, 49

A

abort() (*modules.data.nodes.path.Path* method), 31
 add_gate_type() (*modules.data.ImageBuffer* method), 18
 add_path() (*modules.data.nodes.path.Path* method), 31
 add_paths() (*modules.ui.main_menu.in_progress_view.MainMenuView* method), 78
 add_texture() (*modules.data.ImageBuffer* method), 18
 Adder (class in *modules.data.nodes.eight.addr*), 34
 Adder (class in *modules.data.nodes.one.addr*), 36
 align (*modules.ui.toolbox.text.Text* property), 97
 anchor (*modules.ui.toolbox.button.Button* property), 84
 anchor (*modules.ui.toolbox.hitbox.HitBox* property), 95
 And (class in *modules.data.nodes.eight.gand*), 34
 And (class in *modules.data.nodes.one.gand*), 37
 apply_key() (in module *modules.ui.toolbox.keys*), 96
 Audio (class in *modules.data*), 17

B

back() (*modules.ui.window.Window* method), 49
 BackEaseIn (class in *modules.ui.toolbox.easing*), 85
 BackEaseInOut (class in *modules.ui.toolbox.easing*), 85
 BackEaseOut (class in *modules.ui.toolbox.easing*), 86
 bake_custom_gates() (*modules.data.loader.Loader* method), 28
 bake_level_buttons() (*modules.data.loader.Loader* method), 28
 bake_predefined_gates() (*modules.data.loader.Loader* method), 29
 bake_single_gate() (*modules.data.loader.Loader* method), 29
 bake_textures() (*modules.data.loader.Loader* method), 29
 bottom_bar_update_camera() (*modules.ui.editor.view.EditorView* method), 58
 bottom_bar_update_camera() (*modules.ui.level_player.view.LevelPlayer* method), 74
 bottom_bar_width_sum() (*modules.ui.editor.view.EditorView* method), 58
 bottom_bar_width_sum() (*modules.ui.level_editor.view.LevelEditorView* method), 66
 bottom_bar_width_sum() (*modules.ui.level_player.view.LevelPlayer* method), 74
 bottom_gate_bar() (*modules.ui.editor.view.EditorView* method), 58
 bottom_gate_bar() (*modules.ui.level_editor.view.LevelEditorView* method), 66
 bottom_gate_bar() (*modules.ui.level_player.view.LevelPlayer* method), 74
 BounceEaseIn (class in *modules.ui.toolbox.easing*), 86
 BounceEaseInOut (class in *modules.ui.toolbox.easing*), 86
 BounceEaseOut (class in *modules.ui.toolbox.easing*), 86
 Breaker (class in *modules.data.nodes.mix.eight_breaker*), 36
 buffer (*modules.data.ImageBuffer* attribute), 18
 buffer (*modules.data.LevelButtonsBuffer* attribute), 19
 Button (class in *modules.ui.toolbox.button*), 84

C

calculate_custom() (*modules.engine.logic.Engine* method), 39
 calculate_display() (*modules.data.gate.Gate* method), 23
 calculate_display_lite() (*modules.data.gate.Gate* method), 23
 calculate_inventory() (*modules.data.level.Level* method), 26
 calculate_output() (*modules.engine.logic.Engine* method), 39
 camera (*modules.data.gate.Gate* property), 23
 camera (*modules.ui.editor.view.EditorView* property), 58
 camera (*modules.ui.level_editor.view.LevelEditorView* property), 66
 camera (*modules.ui.level_player.view.LevelPlayer* property), 74
 camera_moving() (*modules.data.gate.Gate* method), 23
 categories (*modules.data.Data* attribute), 18
 check_victory() (*modules.data.level.Level* method),

- 26
- Chip (class in modules.data.chip), 20
- CircularEaseIn (class in modules.ui.toolbox.easing), 87
- CircularEaseInOut (class in modules.ui.toolbox.easing), 87
- CircularEaseOut (class in modules.ui.toolbox.easing), 87
- clean_out_single_branch() (modules.data.nodes.path.Path method), 31
- Clock (class in modules.data.nodes.one.clock), 37
- color (modules.ui.toolbox.button.Button property), 85
- color (modules.ui.toolbox.text.Text property), 97
- COLORS (class in modules.data), 17
- compare_truth_tables() (modules.data.level.Level method), 26
- complete_gate() (modules.data.ImageBuffer method), 19
- Complex (class in modules.data.complex), 21
- copy() (modules.data.chip.Chip method), 20
- CubicEaseIn (class in modules.ui.toolbox.easing), 87
- CubicEaseInOut (class in modules.ui.toolbox.easing), 88
- CubicEaseOut (class in modules.ui.toolbox.easing), 88
- current_lang (modules.data.Data attribute), 18
- CustomGate (class in modules.data.custom), 22
- ## D
- Data (class in modules.data), 17
- debug() (modules.logger.Logger method), 48
- DebugTilesView (class in modules.ui.debug_display_all_tiles.view), 50
- Delay (class in modules.data.nodes.one.delay), 37
- delete() (modules.ui.editor.view.EditorView method), 58
- delete() (modules.ui.level_editor.view.LevelEditorView method), 66
- delete() (modules.ui.level_player.view.LevelPlayer method), 74
- delete_gate() (modules.ui.editor.view.EditorView method), 58
- delete_gate() (modules.ui.level_editor.view.LevelEditorView method), 66
- delete_gate() (modules.ui.level_player.view.LevelPlayer method), 74
- delete_path() (modules.ui.editor.view.EditorView method), 59
- display() (modules.ui.window.Window method), 50
- draw() (modules.data.gate.Gate method), 23
- draw() (modules.data.node.Node method), 31
- draw() (modules.data.nodes.path.Path method), 31
- draw() (modules.ui.toolbox.button.Button method), 85
- draw() (modules.ui.toolbox.entity.Entity method), 94
- draw() (modules.ui.toolbox.grid.Grid method), 95
- draw() (modules.ui.toolbox.hitbox.HitBox method), 95
- draw() (modules.ui.toolbox.poly_hitbox.PolyHitbox method), 96
- draw() (modules.ui.toolbox.text.Text method), 97
- draw_bottom_gates() (modules.ui.editor.view.EditorView method), 59
- draw_bottom_gates() (modules.ui.level_editor.view.LevelEditorView method), 67
- draw_bottom_gates() (modules.ui.level_player.view.LevelPlayer method), 74
- draw_connections() (modules.engine.logic.Engine method), 39
- draw_debug_text() (modules.ui.editor.view.EditorView method), 59
- draw_debug_text() (modules.ui.level_editor.view.LevelEditorView method), 67
- draw_debug_text() (modules.ui.level_player.view.LevelPlayer method), 74
- draw_frame_background() (modules.ui.editor.view.EditorView method), 59
- draw_frame_background() (modules.ui.level_editor.view.LevelEditorView method), 67
- draw_frame_background() (modules.ui.level_player.view.LevelPlayer method), 74
- draw_frame_background() (modules.ui.main_menu.in_progress_view.MainMenuView method), 78
- draw_frame_border() (modules.ui.editor.view.EditorView method), 59
- draw_frame_border() (modules.ui.level_editor.view.LevelEditorView method), 67
- draw_frame_border() (modules.ui.level_player.view.LevelPlayer method), 75
- draw_frame_border() (modules.ui.main_menu.in_progress_view.MainMenuView method), 78
- draw_frame_border_no_bg() (modules.ui.editor.view.EditorView method), 59
- draw_frame_border_top() (modules.ui.level_player.view.LevelPlayer method), 75
- draw_hint() (modules.ui.level_player.view.LevelPlayer method), 75
- draw_level_time() (mod-

ules.ui.level_player.view.LevelPlayer method), 75
 draw_paths() (modules.ui.main_menu.in_progress_view.MainMenuView method), 78
 draw_right() (modules.ui.level_player.view.LevelPlayer method), 75
 draw_tile() (modules.ui.editor.view.EditorView method), 59
 draw_tile() (modules.ui.level_editor.view.LevelEditorView method), 67
 draw_tile() (modules.ui.main_menu.in_progress_view.MainMenuView method), 78
 draw_tiles() (modules.data.complex.Complex method), 21
 draw_tiles() (modules.data.custom.CustomGate method), 22
 draw_tiles() (modules.data.gate.Gate method), 23
 draw_won() (modules.ui.level_player.view.LevelPlayer method), 75

E

EasingBase (class in modules.ui.toolbox.easing), 88
 EditorChipSelector (class in modules.ui.editor.selector), 56
 EditorView (class in modules.ui.editor.view), 58
 ElasticEaseIn (class in modules.ui.toolbox.easing), 89
 ElasticEaseInOut (class in modules.ui.toolbox.easing), 89
 ElasticEaseOut (class in modules.ui.toolbox.easing), 89
 empty (modules.data.nodes.path.Path property), 31
 Engine (class in modules.engine.logic), 39
 English (class in modules.data.language), 25
 Entity (class in modules.ui.toolbox.entity), 94
 error() (modules.logger.Logger method), 48
 ExponentialEaseIn (class in modules.ui.toolbox.easing), 90
 ExponentialEaseInOut (class in modules.ui.toolbox.easing), 90
 ExponentialEaseOut (class in modules.ui.toolbox.easing), 90

F

finish() (modules.data.nodes.path.Path method), 32
 first() (modules.ui.window.Window method), 50
 French (class in modules.data.language), 25
 func() (modules.ui.toolbox.easing.BackEaseIn method), 85
 func() (modules.ui.toolbox.easing.BackEaseInOut method), 86
 func() (modules.ui.toolbox.easing.BackEaseOut method), 86
 func() (modules.ui.toolbox.easing.BounceEaseIn method), 86
 func() (modules.ui.toolbox.easing.BounceEaseInOut method), 86
 func() (modules.ui.toolbox.easing.BounceEaseOut method), 87
 func() (modules.ui.toolbox.easing.CircularEaseIn method), 87
 func() (modules.ui.toolbox.easing.CircularEaseInOut method), 87
 func() (modules.ui.toolbox.easing.CircularEaseOut method), 87
 func() (modules.ui.toolbox.easing.CubicEaseIn method), 88
 func() (modules.ui.toolbox.easing.CubicEaseInOut method), 88
 func() (modules.ui.toolbox.easing.CubicEaseOut method), 88
 func() (modules.ui.toolbox.easing.EasingBase method), 88
 func() (modules.ui.toolbox.easing.ElasticEaseIn method), 89
 func() (modules.ui.toolbox.easing.ElasticEaseInOut method), 89
 func() (modules.ui.toolbox.easing.ElasticEaseOut method), 90
 func() (modules.ui.toolbox.easing.ExponentialEaseIn method), 90
 func() (modules.ui.toolbox.easing.ExponentialEaseInOut method), 90
 func() (modules.ui.toolbox.easing.ExponentialEaseOut method), 90
 func() (modules.ui.toolbox.easing.LinearInOut method), 91
 func() (modules.ui.toolbox.easing.QuadEaseIn method), 91
 func() (modules.ui.toolbox.easing.QuadEaseInOut method), 91
 func() (modules.ui.toolbox.easing.QuadEaseOut method), 92
 func() (modules.ui.toolbox.easing.QuarticEaseIn method), 92
 func() (modules.ui.toolbox.easing.QuarticEaseInOut method), 92
 func() (modules.ui.toolbox.easing.QuarticEaseOut method), 92
 func() (modules.ui.toolbox.easing.QuinticEaseIn method), 93
 func() (modules.ui.toolbox.easing.QuinticEaseInOut method), 93
 func() (modules.ui.toolbox.easing.QuinticEaseOut method), 93
 func() (modules.ui.toolbox.easing.SineEaseIn method), 93
 func() (modules.ui.toolbox.easing.SineEaseInOut method), 94

`func()` (*modules.ui.toolbox.easing.SineEaseOut method*), 94

G

`GameView` (*class in modules.ui.main_menu.view_menu*), 82

`Gate` (*class in modules.data.gate*), 23

`gate_8adder()` (*in module modules.engine.logic*), 41

`gate_8and()` (*in module modules.engine.logic*), 42

`gate_8breaker()` (*in module modules.engine.logic*), 42

`gate_8maker()` (*in module modules.engine.logic*), 42

`gate_8mux()` (*in module modules.engine.logic*), 42

`gate_8nand()` (*in module modules.engine.logic*), 43

`gate_8nor()` (*in module modules.engine.logic*), 43

`gate_8not()` (*in module modules.engine.logic*), 43

`gate_8or()` (*in module modules.engine.logic*), 43

`gate_8subtractor()` (*in module modules.engine.logic*), 44

`gate_8xor()` (*in module modules.engine.logic*), 44

`gate_adder()` (*in module modules.engine.logic*), 44

`gate_and()` (*in module modules.engine.logic*), 45

`gate_clk()` (*in module modules.engine.logic*), 45

`gate_delay()` (*in module modules.engine.logic*), 45

`gate_nand()` (*in module modules.engine.logic*), 45

`gate_nor()` (*in module modules.engine.logic*), 46

`gate_not()` (*in module modules.engine.logic*), 46

`gate_or()` (*in module modules.engine.logic*), 46

`gate_pass()` (*in module modules.engine.logic*), 46

`gate_register()` (*in module modules.engine.logic*), 47

`gate_subtractor()` (*in module modules.engine.logic*), 47

`gate_tand()` (*in module modules.engine.logic*), 47

`gate_tor()` (*in module modules.engine.logic*), 47

`gate_xor()` (*in module modules.engine.logic*), 48

`gen_tile_pattern()` (*modules.data.gate.Gate method*), 24

`generate_thick_line_polygon()` (*modules.data.nodes.path.Path method*), 32

`get()` (*modules.data.language.English method*), 25

`get()` (*modules.data.language.French method*), 25

`get()` (*modules.data.LevelButtonsBuffer method*), 19

`get_connected_points()` (*modules.data.nodes.path.Path method*), 32

`get_gates()` (*modules.data.chip.Chip method*), 20

`get_gates()` (*modules.data.level.Level method*), 26

`get_hovered_bottom_gate()` (*modules.ui.editor.view.EditorView method*), 59

`get_hovered_bottom_gate()` (*modules.ui.level_editor.view.LevelEditorView method*), 67

`get_hovered_bottom_gate()` (*modules.ui.level_player.view.LevelPlayer method*), 75

`get_inputs()` (*modules.data.chip.Chip method*), 20

`get_inputs()` (*modules.data.level.Level method*), 26

`get_outputs()` (*modules.data.chip.Chip method*), 21

`get_outputs()` (*modules.data.level.Level method*), 27

`get_save_gate_counts()` (*modules.ui.level_editor.save.SaveFrame method*), 61

`get_single_truth_table_complex()` (*modules.data.level.Level method*), 27

`get_single_truth_table_simple()` (*modules.data.level.Level method*), 27

`get_stars_count()` (*modules.data.level.Level method*), 27

`get_texture()` (*modules.data.ImageBuffer method*), 19

`get_touched_branch` (*modules.data.nodes.path.Path property*), 32

`get_truth_table()` (*modules.data.level.Level method*), 27

`get_wired_inputs_map()` (*modules.engine.logic.Engine method*), 40

`Grid` (*class in modules.ui.toolbox.grid*), 95

H

`height` (*modules.ui.toolbox.button.Button property*), 85

`height` (*modules.ui.toolbox.entity.Entity property*), 94

`height` (*modules.ui.toolbox.hitbox.HitBox property*), 95

`height` (*modules.ui.toolbox.text.Text property*), 97

`hide()` (*modules.ui.window.Window method*), 50

`Hint` (*class in modules.data.hint*), 24

`HitBox` (*class in modules.ui.toolbox.hitbox*), 95

I

`IMAGE` (*modules.data.Data attribute*), 17

`ImageBuffer` (*class in modules.data*), 18

`info()` (*modules.logger.Logger method*), 49

`Input` (*class in modules.data.nodes.eight.input*), 35

`Input` (*class in modules.data.nodes.one.input*), 37

`InputFrame` (*class in modules.ui.editor.input*), 52

`is_complete_gate()` (*modules.data.ImageBuffer method*), 19

K

`KeyBinds` (*class in modules.data*), 19

L

`launch_next_level()` (*modules.ui.level_player.view.LevelPlayer method*), 75

`Level` (*class in modules.data.level*), 25

`LEVEL_BUTTONS` (*modules.data.Data attribute*), 17

`level_colors` (*modules.data.Data attribute*), 18

`LevelButtonsBuffer` (*class in modules.data*), 19

`LevelEditorSelector` (*class in modules.ui.level_editor.selector*), 63

LevelEditorView (class in modules.ui.level_editor.view), 66
 LevelList (class in modules.ui.level_list.view), 69
 LevelPlayer (class in modules.ui.level_player.view), 74
 LevelPlayerSelector (class in modules.ui.level_player.selector), 71
 LinearInOut (class in modules.ui.toolbox.easing), 91
 load() (modules.data.chip.Chip method), 21
 load() (modules.data.custom.CustomGate method), 22
 load() (modules.data.Data method), 18
 load() (modules.data.gate.Gate method), 24
 load() (modules.data.level.Level method), 28
 load() (modules.data.loader.Loader method), 29
 load() (modules.data.nodes.path.Path method), 32
 load_files() (modules.data.loader.Loader method), 29
 load_fonts() (modules.data.loader.Loader method), 29
 load_hitboxes() (modules.data.nodes.path.Path method), 32
 load_levels() (modules.data.loader.Loader method), 30
 load_saves() (modules.data.loader.Loader method), 30
 load_saves_dependency() (modules.data.loader.Loader method), 30
 load_saves_dependency_round() (modules.data.loader.Loader method), 30
 load_single_chip() (modules.data.loader.Loader method), 30
 load_tilesets() (modules.data.loader.Loader method), 30
 load_tilesets() (modules.ui.debug_display_all_tiles.view.DebugTilesView method), 50
 load_ui() (modules.data.loader.Loader method), 30
 loaded_chips (modules.data.Data attribute), 18
 loaded_levels (modules.data.Data attribute), 18
 Loader (class in modules.data.loader), 28
 Logger (class in modules.logger), 48

M

MainMenuView (class in modules.ui.main_menu.in_progress.view), 78
 Maker (class in modules.data.nodes.mix.eight_maker), 36
 merge() (modules.data.nodes.path.Path method), 33
 module
 modules.data, 17
 modules.data.chip, 20
 modules.data.complex, 21
 modules.data.custom, 22
 modules.data.gate, 23
 modules.data.gate_index, 23
 modules.data.hint, 24

modules.data.language, 25
 modules.data.level, 25
 modules.data.loader, 28
 modules.data.node, 31
 modules.data.nodes.eight.addr, 34
 modules.data.nodes.eight.gand, 34
 modules.data.nodes.eight.gnot, 34
 modules.data.nodes.eight.gor, 34
 modules.data.nodes.eight.input, 35
 modules.data.nodes.eight.nand, 35
 modules.data.nodes.eight.nor, 35
 modules.data.nodes.eight.one, 35
 modules.data.nodes.eight.output, 36
 modules.data.nodes.eight.sub, 36
 modules.data.nodes.eight.xor, 36
 modules.data.nodes.mix.eight_breaker, 36
 modules.data.nodes.mix.eight_maker, 36
 modules.data.nodes.one.addr, 36
 modules.data.nodes.one.clock, 37
 modules.data.nodes.one.delay, 37
 modules.data.nodes.one.gand, 37
 modules.data.nodes.one.gnot, 37
 modules.data.nodes.one.gor, 37
 modules.data.nodes.one.gpass, 37
 modules.data.nodes.one.input, 37
 modules.data.nodes.one.nand, 37
 modules.data.nodes.one.nor, 38
 modules.data.nodes.one.off, 38
 modules.data.nodes.one.on, 38
 modules.data.nodes.one.output, 38
 modules.data.nodes.one.sub, 38
 modules.data.nodes.one.swc, 38
 modules.data.nodes.one.tand, 38
 modules.data.nodes.one.tor, 39
 modules.data.nodes.one.xor, 39
 modules.data.nodes.path, 31
 modules.engine, 39
 modules.engine.logic, 39
 modules.logger, 48
 modules.ui, 49
 modules.ui.debug_display_all_tiles.view, 50
 modules.ui.editor.input, 52
 modules.ui.editor.save, 54
 modules.ui.editor.selector, 56
 modules.ui.editor.view, 58
 modules.ui.level_editor.save, 61
 modules.ui.level_editor.selector, 63
 modules.ui.level_editor.view, 66
 modules.ui.level_list.view, 69
 modules.ui.level_player.selector, 71
 modules.ui.level_player.view, 74
 modules.ui.main_menu.in_progress.view, 78
 modules.ui.main_menu.settings_view, 80

- modules.ui.main_menu.tutorial_view, 81
- modules.ui.main_menu.verification, 82
- modules.ui.main_menu.view_menu, 82
- modules.ui.mouse, 49
- modules.ui.toolbox.button, 84
- modules.ui.toolbox.easing, 85
- modules.ui.toolbox.entity, 94
- modules.ui.toolbox.grid, 95
- modules.ui.toolbox.hitbox, 95
- modules.ui.toolbox.id_generator, 95
- modules.ui.toolbox.keys, 96
- modules.ui.toolbox.poly_hitbox, 96
- modules.ui.toolbox.text, 96
- modules.ui.window, 49
- modules.data
 - module, 17
- modules.data.chip
 - module, 20
- modules.data.complex
 - module, 21
- modules.data.custom
 - module, 22
- modules.data.gate
 - module, 23
- modules.data.gate_index
 - module, 23
- modules.data.hint
 - module, 24
- modules.data.language
 - module, 25
- modules.data.level
 - module, 25
- modules.data.loader
 - module, 28
- modules.data.node
 - module, 31
- modules.data.nodes.eight.addr
 - module, 34
- modules.data.nodes.eight.gand
 - module, 34
- modules.data.nodes.eight.gnot
 - module, 34
- modules.data.nodes.eight.gor
 - module, 34
- modules.data.nodes.eight.input
 - module, 35
- modules.data.nodes.eight.nand
 - module, 35
- modules.data.nodes.eight.nor
 - module, 35
- modules.data.nodes.eight.one
 - module, 35
- modules.data.nodes.eight.output
 - module, 36
- modules.data.nodes.eight.sub
 - module, 36
- modules.data.nodes.eight.xor
 - module, 36
- modules.data.nodes.mix.eight_breaker
 - module, 36
- modules.data.nodes.mix.eight_maker
 - module, 36
- modules.data.nodes.one.addr
 - module, 36
- modules.data.nodes.one.clock
 - module, 37
- modules.data.nodes.one.delay
 - module, 37
- modules.data.nodes.one.gand
 - module, 37
- modules.data.nodes.one.gnot
 - module, 37
- modules.data.nodes.one.gor
 - module, 37
- modules.data.nodes.one.gpass
 - module, 37
- modules.data.nodes.one.input
 - module, 37
- modules.data.nodes.one.nand
 - module, 37
- modules.data.nodes.one.nor
 - module, 38
- modules.data.nodes.one.off
 - module, 38
- modules.data.nodes.one.on
 - module, 38
- modules.data.nodes.one.output
 - module, 38
- modules.data.nodes.one.sub
 - module, 38
- modules.data.nodes.one.swc
 - module, 38
- modules.data.nodes.one.tand
 - module, 38
- modules.data.nodes.one.tor
 - module, 39
- modules.data.nodes.one.xor
 - module, 39
- modules.data.nodes.path
 - module, 31
- modules.engine
 - module, 39
- modules.engine.logic
 - module, 39
- modules.logger
 - module, 48
- modules.ui
 - module, 49

modules.ui.debug_display_all_tiles.view
 module, 50
 modules.ui.editor.input
 module, 52
 modules.ui.editor.save
 module, 54
 modules.ui.editor.selector
 module, 56
 modules.ui.editor.view
 module, 58
 modules.ui.level_editor.save
 module, 61
 modules.ui.level_editor.selector
 module, 63
 modules.ui.level_editor.view
 module, 66
 modules.ui.level_list.view
 module, 69
 modules.ui.level_player.selector
 module, 71
 modules.ui.level_player.view
 module, 74
 modules.ui.main_menu.in_progress_view
 module, 78
 modules.ui.main_menu.settings_view
 module, 80
 modules.ui.main_menu.tutorial_view
 module, 81
 modules.ui.main_menu.verification
 module, 82
 modules.ui.main_menu.view_menu
 module, 82
 modules.ui.mouse
 module, 49
 modules.ui.toolbox.button
 module, 84
 modules.ui.toolbox.easing
 module, 85
 modules.ui.toolbox.entity
 module, 94
 modules.ui.toolbox.grid
 module, 95
 modules.ui.toolbox.hitbox
 module, 95
 modules.ui.toolbox.id_generator
 module, 95
 modules.ui.toolbox.keys
 module, 96
 modules.ui.toolbox.poly_hitbox
 module, 96
 modules.ui.toolbox.text
 module, 96
 modules.ui.window
 module, 49

moving (*modules.data.gate.Gate* property), 24
 multiline (*modules.ui.toolbox.text.Text* property), 97

N

name (*modules.data.gate.Gate* property), 24
 name (*modules.ui.toolbox.button.Button* property), 85
 Nand (*class in modules.data.nodes.eight.nand*), 35
 Nand (*class in modules.data.nodes.one.nand*), 37
 Node (*class in modules.data.node*), 31
 Nor (*class in modules.data.nodes.eight.nor*), 35
 Nor (*class in modules.data.nodes.one.nor*), 38
 Not (*class in modules.data.nodes.eight.gnot*), 34
 Not (*class in modules.data.nodes.one.gnot*), 37

O

Off (*class in modules.data.nodes.one.off*), 38
 On (*class in modules.data.nodes.one.on*), 38
 on_draw() (*modules.ui.debug_display_all_tiles.view.DebugTilesView* method), 50
 on_draw() (*modules.ui.editor.input.InputFrame* method), 52
 on_draw() (*modules.ui.editor.save.SaveFrame* method), 54
 on_draw() (*modules.ui.editor.selector.EditorChipSelector* method), 56
 on_draw() (*modules.ui.editor.view.EditorView* method), 59
 on_draw() (*modules.ui.level_editor.save.SaveFrame* method), 62
 on_draw() (*modules.ui.level_editor.selector.LevelEditorSelector* method), 64
 on_draw() (*modules.ui.level_editor.view.LevelEditorView* method), 68
 on_draw() (*modules.ui.level_list.view.LevelList* method), 70
 on_draw() (*modules.ui.level_player.selector.LevelPlayerSelector* method), 71
 on_draw() (*modules.ui.level_player.view.LevelPlayer* method), 75
 on_draw() (*modules.ui.main_menu.in_progress_view.MainMenuView* method), 78
 on_draw() (*modules.ui.main_menu.settings_view.SettingView* method), 80
 on_draw() (*modules.ui.main_menu.tutorial_view.TutorialView* method), 81
 on_draw() (*modules.ui.main_menu.view_menu.GameView* method), 82
 on_key_press() (*modules.ui.debug_display_all_tiles.view.DebugTilesView* method), 50
 on_key_press() (*modules.ui.editor.input.InputFrame* method), 52
 on_key_press() (*modules.ui.editor.save.SaveFrame* method), 54

<code>on_key_press()</code> <code>ules.ui.editor.selector.EditorChipSelector</code> <code>method)</code> , 56	<code>ules.ui.level_editor.view.LevelEditorView</code> <code>method)</code> , 68
<code>on_key_press()</code> (<code>modules.ui.editor.view.EditorView</code> <code>method)</code> , 60	<code>on_key_release()</code> (<code>mod-</code> <code>ules.ui.level_list.view.LevelList</code> <code>method)</code> , 70
<code>on_key_press()</code> (<code>mod-</code> <code>ules.ui.level_editor.save.SaveFrame</code> <code>method)</code> , 62	<code>on_key_release()</code> (<code>mod-</code> <code>ules.ui.level_player.selector.LevelPlayerSelector</code> <code>method)</code> , 72
<code>on_key_press()</code> (<code>mod-</code> <code>ules.ui.level_editor.selector.LevelEditorSelector</code> <code>method)</code> , 64	<code>on_key_release()</code> (<code>mod-</code> <code>ules.ui.level_player.view.LevelPlayer</code> <code>method)</code> , 76
<code>on_key_press()</code> (<code>mod-</code> <code>ules.ui.level_editor.view.LevelEditorView</code> <code>method)</code> , 68	<code>on_key_release()</code> (<code>mod-</code> <code>ules.ui.main_menu.view_menu.GameView</code> <code>method)</code> , 83
<code>on_key_press()</code> (<code>modules.ui.level_list.view.LevelList</code> <code>method)</code> , 70	<code>on_mouse_motion()</code> (<code>mod-</code> <code>ules.ui.debug_display_all_tiles.view.DebugTilesView</code> <code>method)</code> , 51
<code>on_key_press()</code> (<code>mod-</code> <code>ules.ui.level_player.selector.LevelPlayerSelector</code> <code>method)</code> , 72	<code>on_mouse_motion()</code> (<code>mod-</code> <code>ules.ui.editor.input.InputFrame</code> <code>method)</code> , 53
<code>on_key_press()</code> (<code>mod-</code> <code>ules.ui.level_player.view.LevelPlayer</code> <code>method)</code> , 75	<code>on_mouse_motion()</code> (<code>mod-</code> <code>ules.ui.editor.save.SaveFrame</code> <code>method)</code> , 54
<code>on_key_press()</code> (<code>mod-</code> <code>ules.ui.main_menu.in_progress_view.MainMenuView</code> <code>method)</code> , 78	<code>on_mouse_motion()</code> (<code>mod-</code> <code>ules.ui.editor.selector.EditorChipSelector</code> <code>method)</code> , 56
<code>on_key_press()</code> (<code>mod-</code> <code>ules.ui.main_menu.settings_view.SettingView</code> <code>method)</code> , 80	<code>on_mouse_motion()</code> (<code>mod-</code> <code>ules.ui.editor.view.EditorView</code> <code>method)</code> , 60
<code>on_key_press()</code> (<code>mod-</code> <code>ules.ui.main_menu.tutorial_view.TutorialView</code> <code>method)</code> , 81	<code>on_mouse_motion()</code> (<code>mod-</code> <code>ules.ui.level_editor.save.SaveFrame</code> <code>method)</code> , 62
<code>on_key_press()</code> (<code>mod-</code> <code>ules.ui.main_menu.view_menu.GameView</code> <code>method)</code> , 83	<code>on_mouse_motion()</code> (<code>mod-</code> <code>ules.ui.level_editor.selector.LevelEditorSelector</code> <code>method)</code> , 64
<code>on_key_release()</code> (<code>mod-</code> <code>ules.ui.debug_display_all_tiles.view.DebugTilesView</code> <code>method)</code> , 51	<code>on_mouse_motion()</code> (<code>mod-</code> <code>ules.ui.level_editor.view.LevelEditorView</code> <code>method)</code> , 68
<code>on_key_release()</code> (<code>mod-</code> <code>ules.ui.editor.input.InputFrame</code> <code>method)</code> , 52	<code>on_mouse_motion()</code> (<code>mod-</code> <code>ules.ui.level_list.view.LevelList</code> <code>method)</code> , 70
<code>on_key_release()</code> (<code>modules.ui.editor.save.SaveFrame</code> <code>method)</code> , 54	<code>on_mouse_motion()</code> (<code>mod-</code> <code>ules.ui.level_player.selector.LevelPlayerSelector</code> <code>method)</code> , 72
<code>on_key_release()</code> (<code>mod-</code> <code>ules.ui.editor.selector.EditorChipSelector</code> <code>method)</code> , 56	<code>on_mouse_motion()</code> (<code>mod-</code> <code>ules.ui.level_player.view.LevelPlayer</code> <code>method)</code> , 76
<code>on_key_release()</code> (<code>modules.ui.editor.view.EditorView</code> <code>method)</code> , 60	<code>on_mouse_motion()</code> (<code>mod-</code> <code>ules.ui.main_menu.in_progress_view.MainMenuView</code> <code>method)</code> , 79
<code>on_key_release()</code> (<code>mod-</code> <code>ules.ui.level_editor.save.SaveFrame</code> <code>method)</code> , 62	<code>on_mouse_motion()</code> (<code>mod-</code> <code>ules.ui.main_menu.settings_view.SettingView</code> <code>method)</code> , 80
<code>on_key_release()</code> (<code>mod-</code> <code>ules.ui.level_editor.selector.LevelEditorSelector</code> <code>method)</code> , 64	<code>on_mouse_motion()</code> (<code>mod-</code> <code>ules.ui.main_menu.tutorial_view.TutorialView</code> <code>method)</code> , 81
<code>on_key_release()</code> (<code>mod-</code>	

<code>on_mouse_motion()</code> (modules.ui.main_menu.view_menu.GameView method), 83	<code>on_mouse_release()</code> (modules.ui.editor.selector.EditorChipSelector method), 57
<code>on_mouse_press()</code> (modules.ui.debug_display_all_tiles.view.DebugTilesView method), 51	<code>on_mouse_release()</code> (modules.ui.editor.view.EditorView method), 60
<code>on_mouse_press()</code> (modules.ui.editor.input.InputFrame method), 53	<code>on_mouse_release()</code> (modules.ui.level_editor.save.SaveFrame method), 63
<code>on_mouse_press()</code> (modules.ui.editor.save.SaveFrame method), 55	<code>on_mouse_release()</code> (modules.ui.level_editor.selector.LevelEditorSelector method), 65
<code>on_mouse_press()</code> (modules.ui.editor.selector.EditorChipSelector method), 57	<code>on_mouse_release()</code> (modules.ui.level_editor.view.LevelEditorView method), 69
<code>on_mouse_press()</code> (modules.ui.editor.view.EditorView method), 60	<code>on_mouse_release()</code> (modules.ui.level_list.view.LevelList method), 70
<code>on_mouse_press()</code> (modules.ui.level_editor.save.SaveFrame method), 63	<code>on_mouse_release()</code> (modules.ui.level_player.selector.LevelPlayerSelector method), 73
<code>on_mouse_press()</code> (modules.ui.level_editor.selector.LevelEditorSelector method), 65	<code>on_mouse_release()</code> (modules.ui.level_player.view.LevelPlayer method), 76
<code>on_mouse_press()</code> (modules.ui.level_editor.view.LevelEditorView method), 68	<code>on_mouse_release()</code> (modules.ui.main_menu.view_menu.GameView method), 84
<code>on_mouse_press()</code> (modules.ui.level_list.view.LevelList method), 70	<code>on_mouse_scroll()</code> (modules.ui.editor.view.EditorView method), 61
<code>on_mouse_press()</code> (modules.ui.level_player.selector.LevelPlayerSelector method), 72	<code>on_mouse_scroll()</code> (modules.ui.level_editor.selector.LevelEditorSelector method), 65
<code>on_mouse_press()</code> (modules.ui.level_player.view.LevelPlayer method), 76	<code>on_mouse_scroll()</code> (modules.ui.level_list.view.LevelList method), 71
<code>on_mouse_press()</code> (modules.ui.main_menu.in_progress_view.MainMenuView method), 79	<code>on_mouse_scroll()</code> (modules.ui.level_player.selector.LevelPlayerSelector method), 73
<code>on_mouse_press()</code> (modules.ui.main_menu.settings_view.SettingView method), 80	<code>on_mouse_scroll()</code> (modules.ui.level_player.view.LevelPlayer method), 77
<code>on_mouse_press()</code> (modules.ui.main_menu.tutorial_view.TutorialView method), 82	<code>on_mouse_scroll()</code> (modules.ui.main_menu.settings_view.SettingView method), 81
<code>on_mouse_press()</code> (modules.ui.main_menu.view_menu.GameView method), 83	<code>on_mouse_scroll()</code> (modules.ui.main_menu.tutorial_view.TutorialView method), 82
<code>on_mouse_release()</code> (modules.ui.debug_display_all_tiles.view.DebugTilesView method), 51	<code>on_update()</code> (modules.ui.debug_display_all_tiles.view.DebugTilesView method), 52
<code>on_mouse_release()</code> (modules.ui.editor.input.InputFrame method), 53	<code>on_update()</code> (modules.ui.editor.input.InputFrame method), 54
<code>on_mouse_release()</code> (modules.ui.editor.save.SaveFrame method), 55	<code>on_update()</code> (modules.ui.editor.save.SaveFrame method), 55
<code>on_mouse_release()</code> (modules.ui.editor.selector.EditorChipSelector method), 57	<code>on_update()</code> (modules.ui.editor.selector.EditorChipSelector method), 57

- on_update() (modules.ui.editor.view.EditorView method), 61
 on_update() (modules.ui.level_editor.save.SaveFrame method), 63
 on_update() (modules.ui.level_editor.selector.LevelEditorSelector method), 65
 on_update() (modules.ui.level_editor.view.LevelEditorView method), 69
 on_update() (modules.ui.level_list.view.LevelListView method), 71
 on_update() (modules.ui.level_player.selector.LevelPlayerSelector method), 73
 on_update() (modules.ui.level_player.view.LevelPlayerView method), 77
 on_update() (modules.ui.main_menu.view_menu.GameView method), 84
 ONE (class in modules.data.nodes.eight.one), 35
 Or (class in modules.data.nodes.eight.gor), 34
 Or (class in modules.data.nodes.one.gor), 37
 Output (class in modules.data.nodes.eight.output), 36
 Output (class in modules.data.nodes.one.output), 38
- ## P
- partial_load() (modules.data.chip.Chip method), 21
 Pass (class in modules.data.nodes.one.gpass), 37
 Path (class in modules.data.nodes.path), 31
 play_mode() (modules.data.level.Level method), 28
 PolyHitbox (class in modules.ui.toolbox.poly_hitbox), 96
 prepare_hint_frame() (modules.ui.level_player.view.LevelPlayerView method), 77
 prepare_right_frame_basic() (modules.ui.level_player.view.LevelPlayerView method), 77
 prepare_won_frame() (modules.ui.level_player.view.LevelPlayerView method), 77
 print() (modules.logger.Logger method), 49
 project_point_onto_segments() (modules.data.nodes.path.Path method), 33
 prop_io() (modules.data.custom.CustomGate method), 22
 propagate_outputs() (modules.engine.logic.Engine method), 40
 propagate_values() (modules.engine.logic.Engine method), 40
- ## Q
- QuadEaseIn (class in modules.ui.toolbox.easing), 91
 QuadEaseInOut (class in modules.ui.toolbox.easing), 91
 QuadEaseOut (class in modules.ui.toolbox.easing), 91
 QuarticEaseIn (class in modules.ui.toolbox.easing), 92
 QuarticEaseInOut (class in modules.ui.toolbox.easing), 92
 QuarticEaseOut (class in modules.ui.toolbox.easing), 92
 QuinticEaseIn (class in modules.ui.toolbox.easing), 92
 QuinticEaseInOut (class in modules.ui.toolbox.easing), 93
 QuinticEaseOut (class in modules.ui.toolbox.easing), 93
- ## R
- rainbow_color() (modules.ui.main_menu.in_progress_view.MainMenuView method), 79
 random_id() (in module modules.ui.toolbox.id_generator), 95
 recalculate_hitbox() (modules.data.nodes.path.Path method), 33
 remove_branch() (modules.data.nodes.path.Path method), 33
 render_gate_image() (modules.data.loader.Loader method), 30
 reset() (modules.ui.editor.input.InputFrame method), 54
 reset() (modules.ui.editor.save.SaveFrame method), 56
 reset() (modules.ui.editor.selector.EditorChipSelector method), 57
 reset() (modules.ui.editor.view.EditorView method), 61
 reset() (modules.ui.level_editor.save.SaveFrame method), 63
 reset() (modules.ui.level_editor.selector.LevelEditorSelector method), 66
 reset() (modules.ui.level_editor.view.LevelEditorView method), 69
 reset() (modules.ui.level_list.view.LevelListView method), 71
 reset() (modules.ui.level_player.selector.LevelPlayerSelector method), 73
 reset() (modules.ui.level_player.view.LevelPlayerView method), 77
 reset() (modules.ui.main_menu.view_menu.GameView method), 84
 reset() (modules.ui.toolbox.easing.EasingBase method), 89
 reset_input_validation() (modules.engine.logic.Engine method), 40
 run() (modules.ui.window.Window method), 50
 run_propagation_loop() (modules.engine.logic.Engine method), 41
- ## S
- save() (modules.data.chip.Chip method), 21
 save() (modules.data.custom.CustomGate method), 22
 save() (modules.data.Data method), 18

- save() (*modules.data.gate.Gate method*), 24
 save() (*modules.data.level.Level method*), 28
 save() (*modules.data.nodes.path.Path method*), 33
 save() (*modules.ui.toolbox.poly_hitbox.PolyHitbox method*), 96
 save_frame() (*modules.ui.level_editor.view.LevelEditorView method*), 69
 save_hitboxes() (*modules.data.nodes.path.Path method*), 34
 SaveFrame (*class in modules.ui.editor.save*), 54
 SaveFrame (*class in modules.ui.level_editor.save*), 61
 set() (*modules.data.LevelButtonsBuffer method*), 20
 SettingView (*class in modules.ui.main_menu.settings_view*), 80
 setup() (*modules.ui.editor.selector.EditorChipSelector method*), 58
 setup() (*modules.ui.level_editor.save.SaveFrame method*), 63
 setup() (*modules.ui.level_editor.selector.LevelEditorSelector method*), 66
 setup() (*modules.ui.level_list.view.LevelList method*), 71
 setup() (*modules.ui.level_player.selector.LevelPlayerSelector method*), 73
 setup_editor_categories() (*modules.ui.editor.view.EditorView method*), 61
 setup_texts() (*modules.data.complex.Complex method*), 22
 simulate() (*modules.ui.editor.view.EditorView method*), 61
 simulate() (*modules.ui.level_editor.view.LevelEditorView method*), 69
 simulate() (*modules.ui.level_player.view.LevelPlayer method*), 77
 SineEaseIn (*class in modules.ui.toolbox.easing*), 93
 SineEaseInOut (*class in modules.ui.toolbox.easing*), 93
 SineEaseOut (*class in modules.ui.toolbox.easing*), 94
 size (*modules.ui.toolbox.text.Text property*), 97
 sort_gates() (*modules.engine.logic.Engine method*), 41
 start_chip() (*modules.data.level.Level method*), 28
 Sub (*class in modules.data.nodes.eight.sub*), 36
 Sub (*class in modules.data.nodes.one.sub*), 38
 success() (*modules.logger.Logger method*), 49
 Swc (*class in modules.data.nodes.one.swc*), 38
 switch() (*modules.data.nodes.eight.input.Input method*), 35
 switch() (*modules.data.nodes.eight.one.ONE method*), 36
 switch() (*modules.data.nodes.one.input.Input method*), 37
 T
 TAnd (*class in modules.data.nodes.one.tand*), 38
 Text (*class in modules.ui.toolbox.text*), 96
 text (*modules.ui.toolbox.button.Button property*), 85
 text (*modules.ui.toolbox.text.Text property*), 97
 tick() (*modules.ui.toolbox.easing.EasingBase method*), 89
 Tor (*class in modules.data.nodes.one.tor*), 39
 touched (*modules.data.gate.Gate property*), 24
 touched (*modules.data.nodes.path.Path property*), 34
 touched (*modules.ui.toolbox.button.Button property*), 85
 touched (*modules.ui.toolbox.entity.Entity property*), 94
 touched (*modules.ui.toolbox.hitbox.HitBox property*), 95
 touched (*modules.ui.toolbox.poly_hitbox.PolyHitbox property*), 96
 touched (*modules.ui.toolbox.text.Text property*), 97
 TutorialView (*class in modules.ui.main_menu.tutorial_view*), 81
 U
 UI_EDITOR_GRID_SIZE (*modules.data.Data attribute*), 17
 update_io() (*modules.data.custom.CustomGate method*), 23
 update_text_position() (*modules.data.complex.Complex method*), 22
 update_text_position() (*modules.data.gate.Gate method*), 24
 update_text_readings() (*modules.data.complex.Complex method*), 22
 V
 VALUE_OFF (*modules.data.COLORS attribute*), 17
 VALUE_ON (*modules.data.COLORS attribute*), 17
 VERSION (*modules.data.Data attribute*), 17
 visual_key() (*in module modules.ui.toolbox.keys*), 96
 W
 warning() (*modules.logger.Logger method*), 49
 width (*modules.ui.toolbox.button.Button property*), 85
 width (*modules.ui.toolbox.entity.Entity property*), 94
 width (*modules.ui.toolbox.hitbox.HitBox property*), 95
 width (*modules.ui.toolbox.text.Text property*), 97
 Window (*class in modules.ui.window*), 49
 window (*modules.data.Data attribute*), 18
 WINDOW_FRAMERATE (*modules.data.Data attribute*), 17
 WINDOW_FULLSCREEN (*modules.data.Data attribute*), 17
 WINDOW_HEIGHT (*modules.data.Data attribute*), 18
 WINDOW_WIDTH (*modules.data.Data attribute*), 18
 won() (*modules.ui.level_player.view.LevelPlayer method*), 77
 X
 x (*modules.data.gate.Gate property*), 24
 x (*modules.ui.toolbox.button.Button property*), 85

x (*modules.ui.toolbox.entity.Entity property*), 94
x (*modules.ui.toolbox.hitbox.HitBox property*), 95
x (*modules.ui.toolbox.text.Text property*), 97
Xor (*class in modules.data.nodes.eight.xor*), 36
Xor (*class in modules.data.nodes.one.xor*), 39

Y

y (*modules.data.gate.Gate property*), 24
y (*modules.ui.toolbox.button.Button property*), 85
y (*modules.ui.toolbox.entity.Entity property*), 94
y (*modules.ui.toolbox.hitbox.HitBox property*), 95
y (*modules.ui.toolbox.text.Text property*), 97